OR/Studio

USER GUIDE OPERATIONS RESEARCH WORKBENCH

OR/Studio user guide

Learn every solver, from typing a problem to reading the answer. Written for engineering students and their teachers.

DOCUMENT	**OR/Studio user guide** · free, step-by-step Operations Research solver		
APP VERSION	**2.0.0** · 13 modules	EDITION	October 2026
COURSE	Operations Research · 702CO0E034 · NMIMS Shirpur		
GUIDE BY	**Dr. Praveenkumar Loharkar**		
LIVE APP	or-studio.antideploy.app	LICENCE	MIT, free to use and share
SOURCE CODE	github.com/Jay-Naik2526/or-studio		

00

00 / HOW TO READ THIS GUIDE

How to read this guide

Who it is for, how it is organised, and a clickable list of everything inside.

SHEET 00 OF 25

USER GUIDE

This guide shows you how to use **OR/Studio**, a free solver for Operations Research. You do not need to know the app. You do not need to install anything. You only need a web browser.

A WHO IT IS FOR

- **Students** who learn Operations Research and want to check their hand work.
- **Teachers** who want to show every step on a screen, or set practice work.
- **Anyone** who needs a quick answer to a planning problem.

B HOW IT IS ORGANISED

- **Chapters 1 to 7** explain the app: how to find your way, enter a problem, learn with it, and save your work.
- **Chapters 8 to 20** cover one module each. All use the same seven parts.
- **Walk-throughs A to C** follow three full problems. **Chapters 21 to 25** are for reference.

C THE SIGNS USED IN THIS GUIDE

`max 3x1 + 5x2`	Text in this style is what you **type** or what the screen **shows**.
← → Space	Keyboard keys. `Ctrl/⌘` means Ctrl on Windows and Linux, and Command on a Mac.
Highlighted words	The most important idea on that line.
TIP WATCH OUT	A tip saves you time. A *watch out* box warns you about a common mistake.
01 Model 02 Solution	The two parts of every workspace. **Model** is what you type. **Solution** is what the app gives back.

Technical words are explained the first time they appear. All short forms (such as RHS, BFS or MODI) are also listed in the glossary in chapter 22. In the app you can also hover over most short forms to see their meaning.

D WHERE TO START

IF YOU…	READ
have never seen the app	Chapters 2, 3 and 4. Then do Walk-through A.
need to solve a problem now	Chapter 5, then the chapter of your module.
want to practise for an exam	Chapter 6 (Tutor mode, Diff mode and the problem library).
teach with the app	Chapter 6 (presentation mode) and chapter 7 (share links, reports).
see a strange message	Chapter 21 and chapter 23.

CONTENTS

Contents

Click any line to jump to it. In the PDF viewer, the bookmarks list also shows every chapter.

SHEET 00
USER GUIDE

THE BASICS

01	What OR/Studio is, and is not	The 13 modules at a glance	4
02	Getting started	Open, install, privacy, theme, phone	6
03	Finding your way	Rail, top bar, search, shortcuts	8
04	The workspace explained	Every part of the screen	10
05	Entering a problem	Text, form, grids, graphs, paste	12
06	Learning tools	Step player, Tutor, Diff, library	16
07	Save, share and export	Autosave, link, PDF, LaTeX, CSV	19

THE 13 MODULES

08	Linear programming	Simplex, Graphical, Sensitivity, Duality	21
09	Integer programming	Branch and bound, Gomory cuts	25
10	Nonlinear programming	Gradient, Newton, KKT, quadratic	27
11	Transportation	NW corner, least cost, Vogel, MODI	29
12	Assignment	Hungarian method	31
13	Network models	Shortest path, spanning tree, max flow	33
14	Project planning	CPM, PERT, crashing	35
15	Queuing analysis	M/M/1, M/M/c, finite queues	37
16	Inventory	EOQ, EPQ, shortages, discounts, newsvendor	39
17	Zero-sum games	Saddle point, dominance, mixed strategies	41
18	Markov chains	n-step, steady state, absorbing	43
19	Simulation	Monte Carlo, queue, demand, inventory, RNG	45
20	Job sequencing	Johnson, CDS, exact, Gantt	47

WORKED WALK-THROUGHS

A	Reddy Mikks, from typing to shadow prices	Linear programming	49
B	Experiment 10, activity 1: demand simulation	Simulation	52
C	Experiment 10, activity 4: Johnson's rule	Job sequencing	54

REFERENCE

21	Reading results	Infeasible, Unbounded, Degenerate…	56
22	Glossary of short forms	273 entries	57
23	Troubleshooting and FAQ	When something looks wrong	64
24	Syllabus map	Course 702CO0E034 to modules	66
25	About	Credits and licence	67

01

01 / WHAT OR/STUDIO IS

What OR/Studio is, and is not

A free workbench that solves Operations Research problems and explains every step.

SHEET 01 OF 25

USER GUIDE

Operations Research (OR) uses maths to make good decisions: how much to make, where to ship, which job to do first, how many staff to hire. **OR/Studio** is a website that solves these problems for you, and **shows how**.

1 WHAT IT IS

- **A solver and a teacher in one.** It gives the answer, then lets you move through the working one step at a time, with a short reason for each step.
- **Exact.** It keeps fractions as fractions: 1/3 stays 1/3 and is never rounded to 0.333. This makes special cases (such as ties and cycling) easy to detect.
- **Free and private.** No account, no server, no payment. Everything runs on your own device. It works offline after the first visit.
- **Made for learning.** A step player, a Tutor mode that asks you to choose each move, a Diff mode that finds your mistake, and 66 worked problems.
- **Open source.** MIT licence. Anyone can read, copy and improve the code.

2 WHAT IT IS NOT

- **Not an industrial solver.** It is made for textbook-size problems (up to a few hundred variables). Use professional software for very large models.
- **Not a replacement for learning.** It shows the method, but you still need to understand the idea. Use it to check your own work.
- **Not always exact.** Linear, integer, transport, assignment, network, game and Markov solvers use exact fractions. Queues, inventory and simulation use ordinary decimal numbers, as is normal for these topics.
- **Not connected to the TORA software.** It follows the same textbook course (Taha, *Operations Research: An Introduction*) but is a separate program.

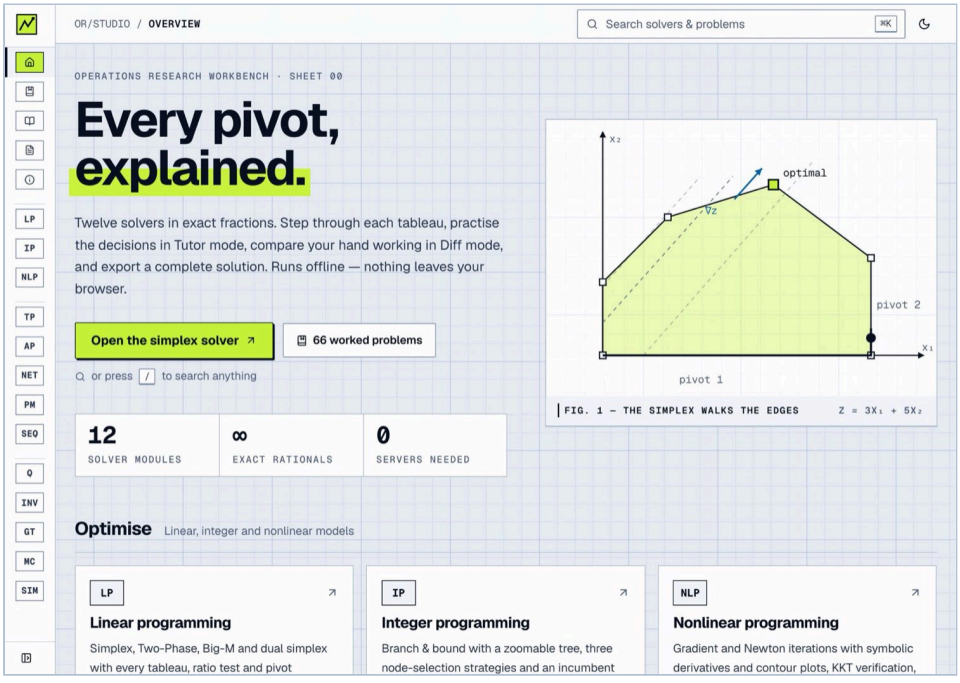


FIG 1.1 · THE OVERVIEW PAGE. THE CARDS LEAD TO THE 13 MODULES, AND THE SEARCH BOX FINDS ANYTHING.

3 THE 13 MODULES AT A GLANCE

MODULE	WHAT IT SOLVES	WHO NEEDS IT
Linear programming	Best mix of products or resources under straight-line limits. Simplex, Two-Phase, Big-M, Dual, Graphical, Sensitivity, Duality.	Product mix, blending and diet problems; Unit 2.
Integer programming	The same, but some answers must be whole numbers or yes/no. Branch and bound, Gomory cuts.	Anything counted in whole units: machines, projects, items in a bag.
Nonlinear programming	Best value of a curved function. Gradient, Newton, KKT check, quadratic programs.	Optimisation with curves; advanced students.
Transportation	Cheapest way to ship goods from sources to destinations. NW corner, least cost, Vogel, MODI.	Logistics and supply planning; Unit 4.
Assignment	Best one-to-one match of workers to jobs. Hungarian method.	Task allocation, timetables; Unit 4.
Network models	Shortest route, cheapest spanning tree, maximum flow with minimum cut.	Roads, pipes, cables, data networks.
Project planning	Project length and critical path. CPM, PERT, time-cost crashing, Gantt chart.	Project managers and construction; Unit 5.
Queuing analysis	Waiting lines: waiting time, queue length, number of servers.	Banks, call centres, service desks.
Inventory	How much to order and when. EOQ, production lots, shortages, discounts, newsvendor.	Stores, factories, supply chains.
Zero-sum games	Best strategies for two opponents. Saddle point, dominance, mixed strategies.	Competition and decision making; Unit 6.
Markov chains	Systems that move between states by chance: n-step, steady state, absorbing chains.	Weather, brand switching, reliability.
Simulation	Learning from random trials: Monte Carlo, queue, demand table, inventory, random-number lab.	Systems too hard for a formula; Unit 7 and Experiment 10.
Job sequencing	Best order of jobs on 2, 3 or more machines. Johnson, CDS, exact search, Gantt chart.	Workshops and production lines; Unit 7 and Experiment 10.

02

02 / GETTING STARTED

Getting started

Open the app, use it offline, keep your work private, and use it on a phone.

SHEET
02 OF 25
USER GUIDE

1 OPEN IT IN ANY BROWSER

1. Open a modern browser: Chrome, Edge, Firefox, Safari or Brave.
2. Go to **or-studio.antideploy.app**. The overview page opens.
3. Click a module card, or press `/` and type what you want. Try `reddy`.

There is nothing to install and no sign-in. The first visit needs the internet.

2 USE IT OFFLINE

After your first visit, the app saves itself in the browser. Next time it opens even without the internet. When you are online again, the app checks for a newer version each time you open it. See the FAQ to force an update.

> **TIP**
> Before an exam or a trip, open the app once while you are online. Then you can use it on a plane or in a lab with no network.

3 INSTALL IT AS AN APP

OR/Studio is a *progressive web app*. That means your browser can put it on your desktop or home screen, so it opens in its own window like any other program.

Chrome, Edge, Brave (computer)	Click the **install** icon at the right end of the address bar, or open the browser menu and choose **Install OR-Studio**.
Android (Chrome)	Open the menu and choose **Install app** or **Add to Home screen**.
iPhone or iPad (Safari)	Tap **Share**, then **Add to Home Screen**.

The exact words depend on your browser version. If you do not see an install option, you can keep using the website normally.

4 YOUR WORK STAYS ON YOUR DEVICE

- OR/Studio has **no server**. The solving happens inside your browser. Your problems are never sent anywhere.
- Your work is saved in your browser's own storage on this device only. Clearing the browser's site data removes it.
- A **share link** holds the whole model inside the link itself. Anyone who has the link can open your model, so share it only with people you trust.

5 LIGHT AND DARK THEME

Use the **moon / sun button** in the top bar to switch. The app opens in the light theme and remembers your choice. You can also open the search box (`/`) and run **Switch light / dark theme**. Printed reports always use a clean light layout.

6 USING IT ON A PHONE

- The left rail turns into a **menu button** (three lines, top right). Tap it to see all modules.
- On a narrow screen the **01 Model** card starts closed to save space. Tap **Show** to type your problem, and **Hide** when done.
- The step player shows **Previous / Play / Next** and the slider. The first/last and speed buttons are hidden to save space.
- On a touch screen, **tap** a short form (such as RHS) to see its meaning.
- Turn the phone sideways to see wide tables.

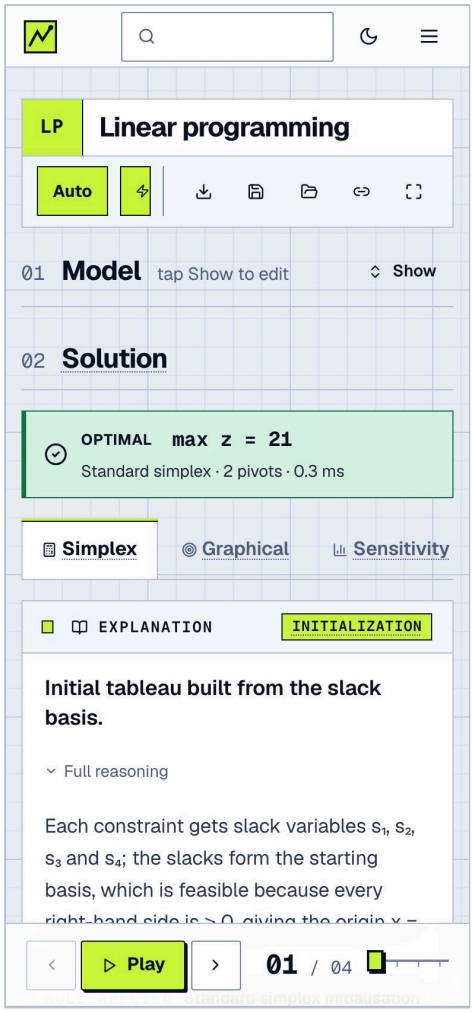


FIG 2.1 · THE WORKSPACE ON A PHONE. THE MODEL IS HIDDEN UNTIL YOU TAP SHOW.

03 / FINDING YOUR WAY

03 Finding your way

The left rail, the top bar, the search box, hover help and keyboard shortcuts.

SHEET 03 OF 25 USER GUIDE

1 THE LEFT RAIL

On a computer, a narrow **rail** runs down the left side. It has two parts.

- **Pages:** Overview (home), Problem library, Manual and About.
- **Modules,** in three groups, each with a short code (LP, IP, NLP…): *Optimise* (LP, IP, NLP), *Flows and plans* (TP, AP, NET, PM, SEQ) and *Uncertainty and decisions* (Q, INV, GT, MC, SIM).

On wide screens (about 1560 pixels and more) the rail shows full names. On smaller screens it shows only the codes. Use **Collapse** or **Expand** at the bottom of the rail to change this. The current page is highlighted in lime.

2 THE TOP BAR

- **Breadcrumb** on the left: `OR/STUDIO / LINEAR PROGRAMMING` shows where you are. Click **OR/STUDIO** to go home.
- **Search box** in the middle-right (see below).
- **Theme button** (moon or sun).
- **Menu button** (phone and small screens only).

3 THE SEARCH PALETTE

The fastest way to move around. Press `/` or `Ctrl/⌘` + `K`, or click the search box. Type a few letters. The list shows pages, modules, methods and worked problems that match. Use `↑` `↓` to choose and `↵` to open. Press `Esc` to close.

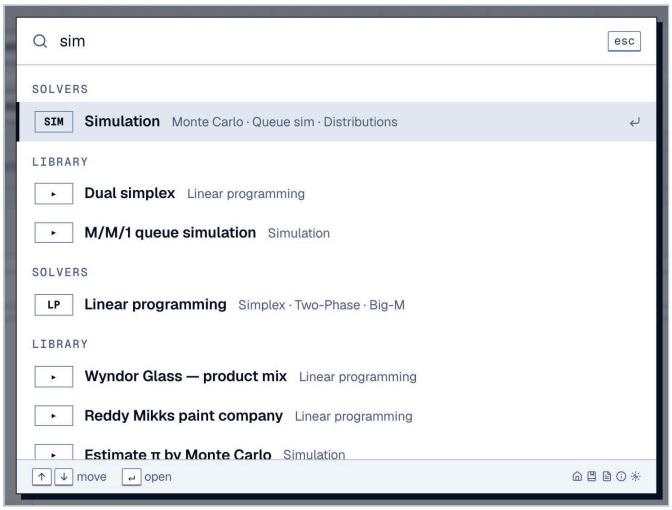


FIG 3.1 · TYPING *SIM* FINDS THE SIMULATION MODULE AND ITS WORKED PROBLEMS.

You can search for:

- a **module**: `queue`, `transport`
- a **method**: `vogel`, `hungarian`, `johnson`
- a **worked problem**: `reddy`, `knapsack`
- an **action**: `theme`

Choosing a worked problem opens its module with the problem already filled in.

> **TIP**
> The `/` key does not work while you are typing in a box. Click outside the box first, or use `Ctrl/⌘` + `K`, which works almost everywhere.

4 HOVER EXPLANATIONS FOR SHORT FORMS

Textbooks are full of short forms. In OR/Studio, move the mouse over a short form, or tap it on a touch screen, or move to it with the keyboard. A small box explains it. Words with a dotted underline have an explanation. This works for column headings (RHS, ES, EF), symbols (ρ, λ), variable names (s_1 is a slack, a_1 is an artificial variable), and method names (VAM, MODI). Press `Esc` to close the box.

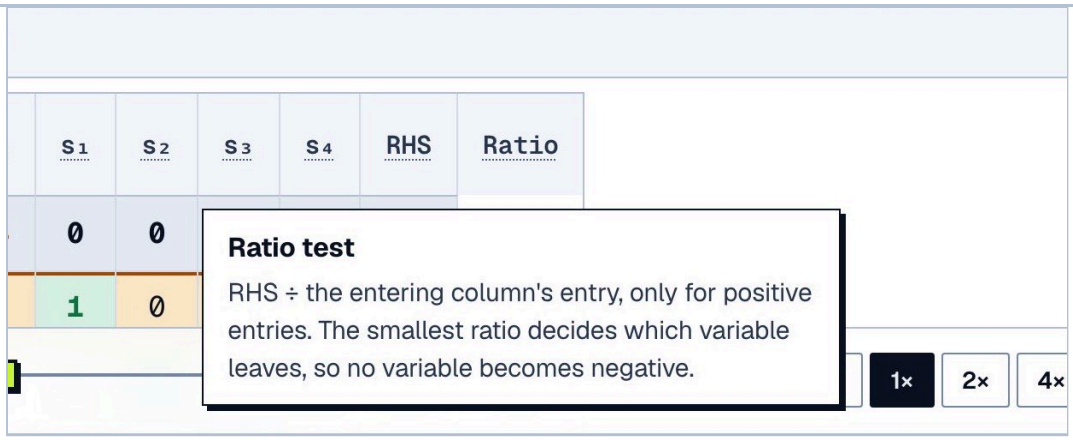


FIG 3.2 · HOVERING OVER *RATIO* IN A TABLEAU SHOWS WHAT THE RATIO TEST IS.

5 KEYBOARD SHORTCUTS

KEY	WHAT IT DOES
`/` or `Ctrl/⌘` `K`	Open the search palette
`←` / `→`	Previous step / next step
`Home` / `End`	First step / last step
`Space`	Play or pause the step player
`1` to `9`	Jump straight to step 1 to 9
`P`	Presentation mode on or off
`Esc`	Close a dialog or menu; leave presentation mode

WATCH OUT

The step keys do nothing while your cursor is inside a text box or a grid cell, because those keys are needed for typing. Click on an empty part of the page first.

04

04 / THE WORKSPACE

The workspace explained

Every module uses the same screen, laid out like a worked sheet. Learn it once and you know all 13.

SHEET
04 OF 25
USER GUIDE

Read the screen from top to bottom. First the **title block**, then **01 Model** (what you type), then **02 Solution** (the answer and the reasons). The numbers on the picture match the list below it.



FIG 4.1 · THE WORKSPACE OF THE LINEAR PROGRAMMING MODULE (REDDY MIKKS PROBLEM, STEP 3 OF 4).

1 **Title block.** The module code (LP), its name and the methods inside it.

2 **Method switch.** Choose how the problem is solved (here Auto, Simplex, Two-Phase, Big-M, Dual). Not every module has one.

3 **Auto / Tutor.** *Auto* solves and shows every step. *Tutor* hides the next move so you can choose it.

4 **Toolbar.** Export, Save, Open, and Copy share link (left to right).

5 **Presentation mode** (also key P). Bigger text for a classroom screen.

6 **01 Model.** Where you enter or load the problem. Anything you change updates the solution at once.

7 **Hide / Show.** Closes or opens the Model card, to give the solution more room.

8 **02 Solution.** Everything the solver found.

9 **Result banner.** Status (such as OPTIMAL), the key number, the method, the number of steps and the time.

10 **Tabs.** Different views of the same answer: tables, graphs, sensitivity, and more.

11 **Explanation.** A short sentence for the current step. Open *Full reasoning* for more.

12 **Step player.** Move through the solution: first, previous, play, next, last, and a slider.

13 **Speed.** How fast Play runs: 0.5×, 1×, 2× or 4×. Hidden on small screens.

4.1 The parts in more detail

1 RESULT BANNER AND STATUS COLOURS

The banner is the first thing to read. It uses colour *and* words, so you never depend on colour alone.

OPTIMAL	The best answer was found. The key number (for example `max z = 21`) is shown next to it.
WARNING KINDS	Something needs your attention, for example an alternate optimum, a degenerate answer, or an iteration limit.
INFEASIBLE / UNBOUNDED	No answer exists, or the answer has no limit. Chapter 21 explains what to do.

2 EXPLANATION CARD

It explains **the step you are looking at**. It has a short sentence (for example *x_2 enters, s_2 leaves. Pivot element = 4/3.*), a **Full reasoning** paragraph, a formula where one helps, and a **Rule applied** line that names the rule. A yellow note appears if something special happened.

3 TABS

Tabs show the same problem in different ways. Their names depend on the module. In Linear programming they are Simplex, Graphical, Sensitivity, Duality, Basic solutions, Primal–dual and Diff mode. Use the mouse, or click a tab and then use `←` `→` to move between tabs.

4 STEP PLAYER

The player is fixed at the bottom of the screen so it is always within reach. See chapter 6. In Tutor mode it is replaced by the Tutor card, because you now decide when to move on.

5 HIDE AND SHOW THE MODEL

Click **Hide** at the right end of the **01 Model** line to close the input and see more of the solution. Click **Show** to open it again. The app remembers your choice.

6 PRESENTATION MODE

Press `P`, or click the last icon in the toolbar. The Model card is hidden, and the writing and tables become larger, so a class at the back of the room can read them. Press `P` or `Esc` to leave.

> **TIP**
> Teachers: load a problem from the library, press `P`, and use `→` to walk the class through the method one step at a time. Fig 6.4 shows the result.

05 / ENTERING A PROBLEM

05 Entering a problem

Type it, fill a form, edit a grid, paste from Excel, or draw a graph.

SHEET 05 OF 25 USER GUIDE

The **01 Model** card changes with the module. This table shows what to expect.

MODULE	HOW YOU ENTER THE PROBLEM
Linear and Integer programming	Type the model as text, or use the **Form** tab (see 5.1 and 5.2).
Nonlinear programming	Type a formula such as `(x1 - 3)^2 + 2*(x2 + 1)^2`, plus a start point or constraints.
Transportation, Assignment, Zero-sum games, Markov chains, Job sequencing	Fill a grid of numbers (see 5.3).
Project planning	A table with one row per activity: name, what comes before it, and times (and costs for crashing).
Network models	Draw the graph with the mouse, or edit the table of edges (see 5.4).
Queuing and Inventory	Boxes for rates and costs. Queuing also has live sliders.
Simulation	Boxes, a distribution table, and a list of random numbers, depending on the kind of simulation.

5.1 The LP text format

Write the model the way you would on paper. The *objective* (what you want to make as large or as small as possible) comes first. The *constraints* (the limits) follow.

```
max 3x1 + 5x2
subject to
  labour: x1 + 2x2 <= 10
  3x1 + 2x2 >= 6
  x1 - x2 = 1
  free x3          (x3 may be negative)
  int x1           (x1 must be a whole number)
  bin x2           (x2 must be 0 or 1)
```

`max` or `min`	Starts the objective. After it, write the constraints under `subject to` (you can also write `st` or `s.t.`).
`<=` `>=` `=`	The relations. You can also type `≤` and `≥`. Variables may be on both sides.
`x1, x2 >= 0`	Variables are already non-negative. Writing this line is allowed but not needed.
`x1` means $1 \cdot x_1$	Coefficients may be left out. Fractions are fine: `1/2x`.
`2 <= x + y <= 8`	A range in one line.
`labour:`	A name for a row. It then appears in the results instead of (1), (2)…
`free` / `int` / `bin`	Declarations: unrestricted sign, whole number, or 0/1 (zero or one).

The line below the box tells you if the text was understood: *2 variables, 4 constraints — parsed*. If something is wrong, the app shows the **line and column** and puts a caret under the problem.

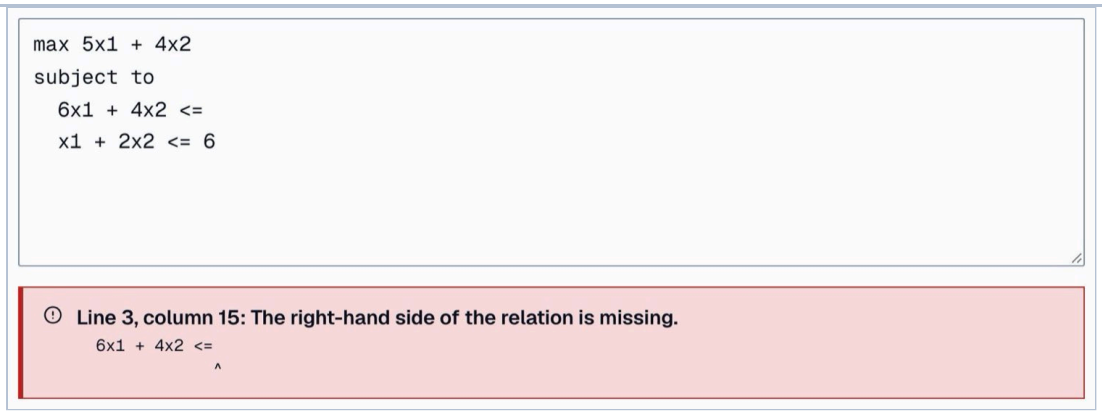


FIG 5.1 · A MISSING RIGHT-HAND SIDE ON LINE 3. THE CARET SHOWS WHERE THE APP STOPPED READING.

WATCH OUT

Always look at the line under the box. If the app cannot read your text, it stops and points at the problem. The old solution stays on screen until the text is valid again. If you are stuck, rewrite the line in its simplest form: coefficient, variable, relation, number.

5.2 The Form tab

Prefer a grid? Click **Form** at the top right of the Model card. It edits the same model. Choose **Maximise** or **Minimise**, add or remove variables with **–** and **+**, type the numbers, pick the relation (≤, ≥ or =) and the right-hand side. The **Type** row sets each variable to ≥ 0, free, integer or binary. Switch back to **Text** at any time. The text is rewritten from the form.

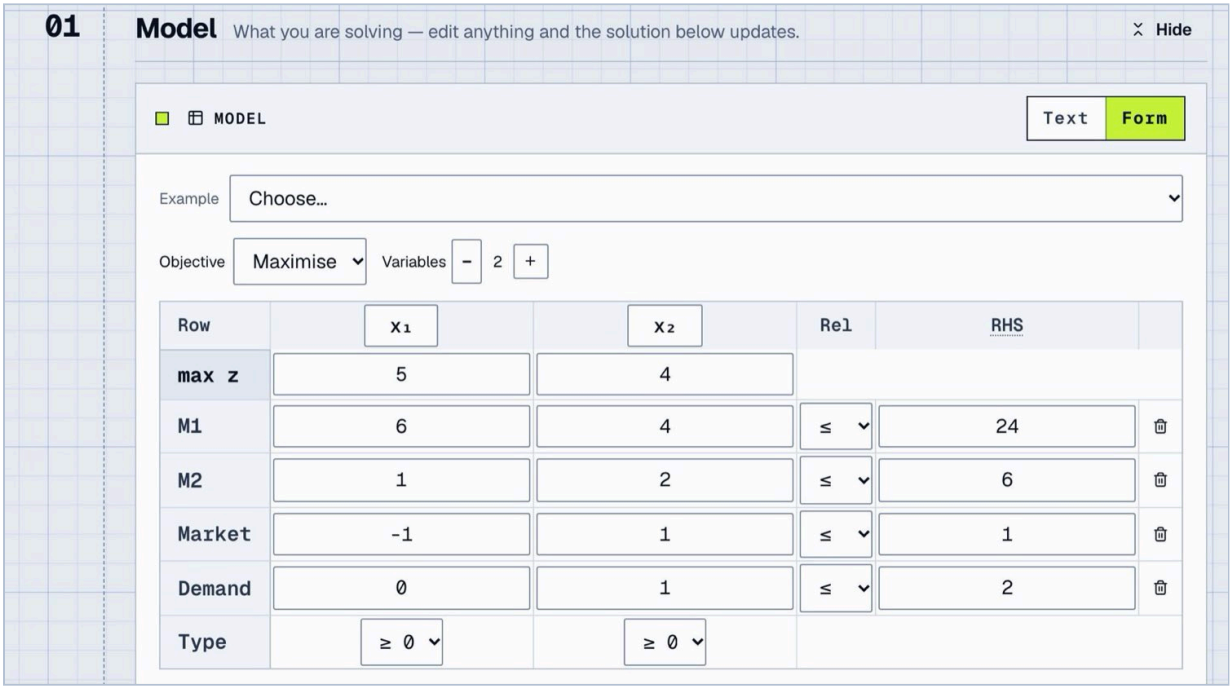


FIG 5.2 · THE FORM TAB FOR THE REDDY MIKKS PROBLEM. EACH ROW IS ONE CONSTRAINT.

5.3 Grids (matrix editors)

Transportation, Assignment, Games, Markov and Job sequencing use a grid. Click a cell and type.

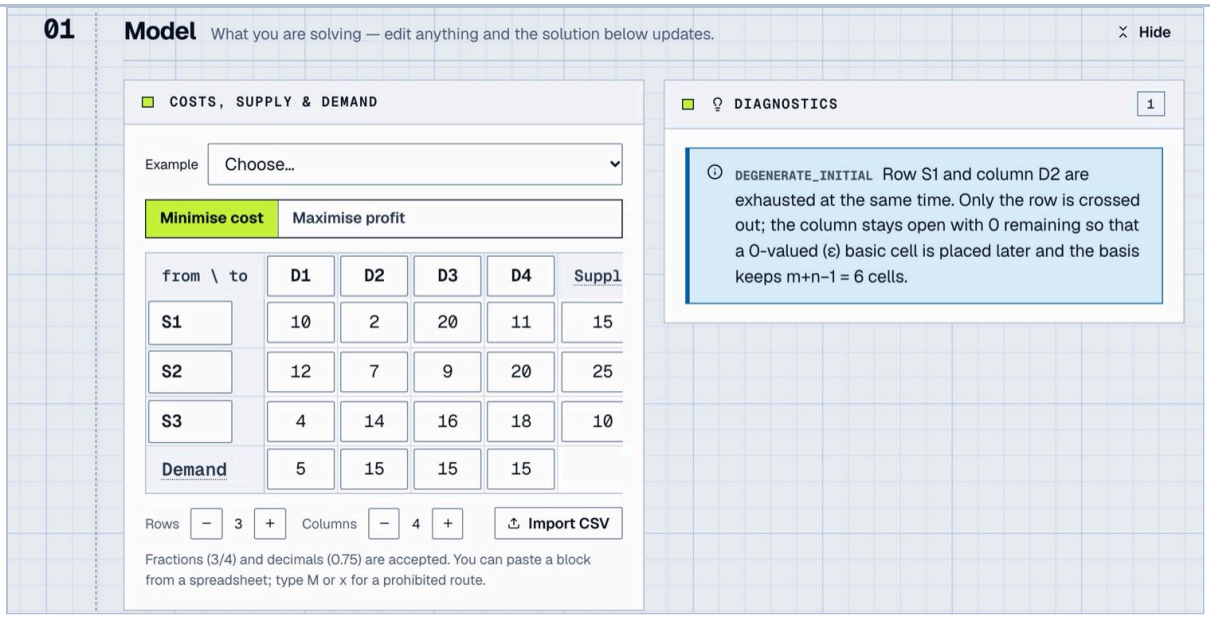


FIG 5.3 · A GRID EDITOR (TRANSPORTATION). ROWS AND COLUMNS ARE ADDED WITH THE + AND − BUTTONS UNDER THE GRID.

- **Move:** `Enter` or `↓` goes down a cell, `↑` goes up. You can also click any cell.
- **Add or remove a row or column:** use the **−** and **+** buttons next to **Rows** and **Columns** under the grid. A square grid (Markov) has one **Size** button. The buttons stop at the largest size the module allows.
- **Names:** the row and column headings are boxes. Click one and type a new name, such as *Plant 1* or *Job A*.
- **Numbers:** whole numbers, decimals (`0.75`) and fractions (`3/4`) are accepted. A cell that is not a number is marked as wrong.
- **Prohibited route:** in Transportation and Assignment, type `M` or `x` in a cell to forbid it.

+ PASTE FROM EXCEL OR A CSV FILE

1. In your spreadsheet, select the block of numbers and copy it (`Ctrl/⌘` + `C`).
2. In OR/Studio, click the **top-left cell** where the block should start.
3. Paste (`Ctrl/⌘` + `V`). The grid grows if the block is larger, up to the module's limit.

You can also click **Import CSV** under the grid and choose a `.csv` , `.tsv` or `.txt` file.

> **TIP**
> Paste only the numbers, not the headings. Then rename the rows and columns in the app.

5.4 The graph editor (Network models)

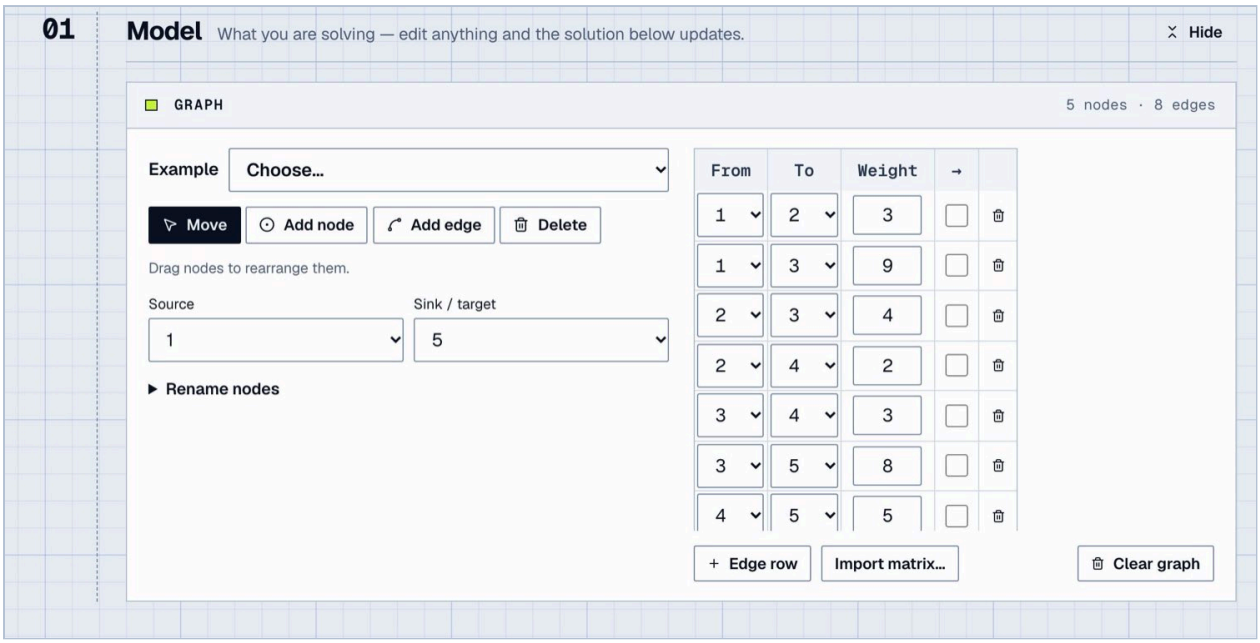


FIG 5.4 · THE GRAPH EDITOR WITH THE SHORTEST-ROUTE EXAMPLE LOADED.

Move	Drag nodes to rearrange the picture. This never changes the answer.
Add node	Choose the tool, then click an empty place on the canvas.
Add edge	Click one node, then another node, to join them.
Delete	Click a node or an edge to remove it.
Edge table	Below the canvas, each edge is a row: **From**, **To**, **Weight** (and **Cap.** for max flow) and a box for **directed** (an arrow, one-way). Edit numbers here, or press **Edge row** to add one.
Source, Sink	Choose the start and end nodes in the two lists above the canvas.
Other	**Rename nodes** (folded section), **Import matrix…** (from a CSV file) and **Clear graph**.

5.5 Examples and Clear

Almost every Model card has an **Example** list at the top. It holds the library problems for that module. Pick one to fill in the model. Some modules also have a clear button (**Clear** in Job sequencing, **Clear graph** in Network models) that empties the problem so you can start from nothing.

> **WATCH OUT**
> Choosing an example **replaces** what is in the Model card. Save your own model first if you want to keep it (see chapter 7). Your last model is also autosaved.

06

06 / LEARNING TOOLS

Learning tools

The step player, Tutor mode, Diff mode, the problem library and presentation mode.

SHEET 06 OF 25
USER GUIDE

6.1 The step player

Every solution is a list of steps, for example one step for each simplex pivot. The player at the bottom of the screen moves you through them.

⏮ ◀ ▶ ⏭	First step, previous step, next step, last step.
Play / Pause	Moves forward by itself. At the end, Play starts again from step 1.
03 / 04	Where you are, and how many steps there are.
Slider	Drag to jump to any step.
0.5× 1× 2× 4×	The speed of Play.

Keys work too: `←` `→` `Home` `End` `Space` and `1` to `9`. The page address also remembers the step, so a link to a step opens at that step.

6.2 Tutor mode

Switch **Auto** to **Tutor** in the title block. The solver now stops before each decision and **asks you**. For example in the simplex method it asks *Which variable should enter the basis?* and then *Which variable leaves?*. Choose an answer.



FIG 6.1 · A WRONG ANSWER. THE APP EXPLAINS WHY, AND NAMES THE IDEA THAT LED YOU ASTRAY.

- A **wrong answer** turns red and explains the specific mistake, with a line called *Misconception*.
- A **right answer** turns green. Press **Continue** for the next question.
- After two wrong answers a **Show the answer** button appears.
- The badge on the card (for example *3/5 first try*) counts questions you got right on the first try.

Tutor mode is in **Linear programming, Integer programming, Transportation, Assignment, Network models, Project planning, Zero-sum games and Job sequencing**. The other modules use formulas and have no decisions to practise. Changing the model or the method restarts the lesson.

6.3 Diff mode

Solve the problem **by hand** first. Then open the **Diff mode** tab and type your working for one step. OR/Studio compares it with the correct working, finds the **first place they differ**, lists the cells, and guesses the cause: a wrong entering or leaving variable, a pivot row that was not divided, a wrong multiplier, a sign error, or a single slip in arithmetic.

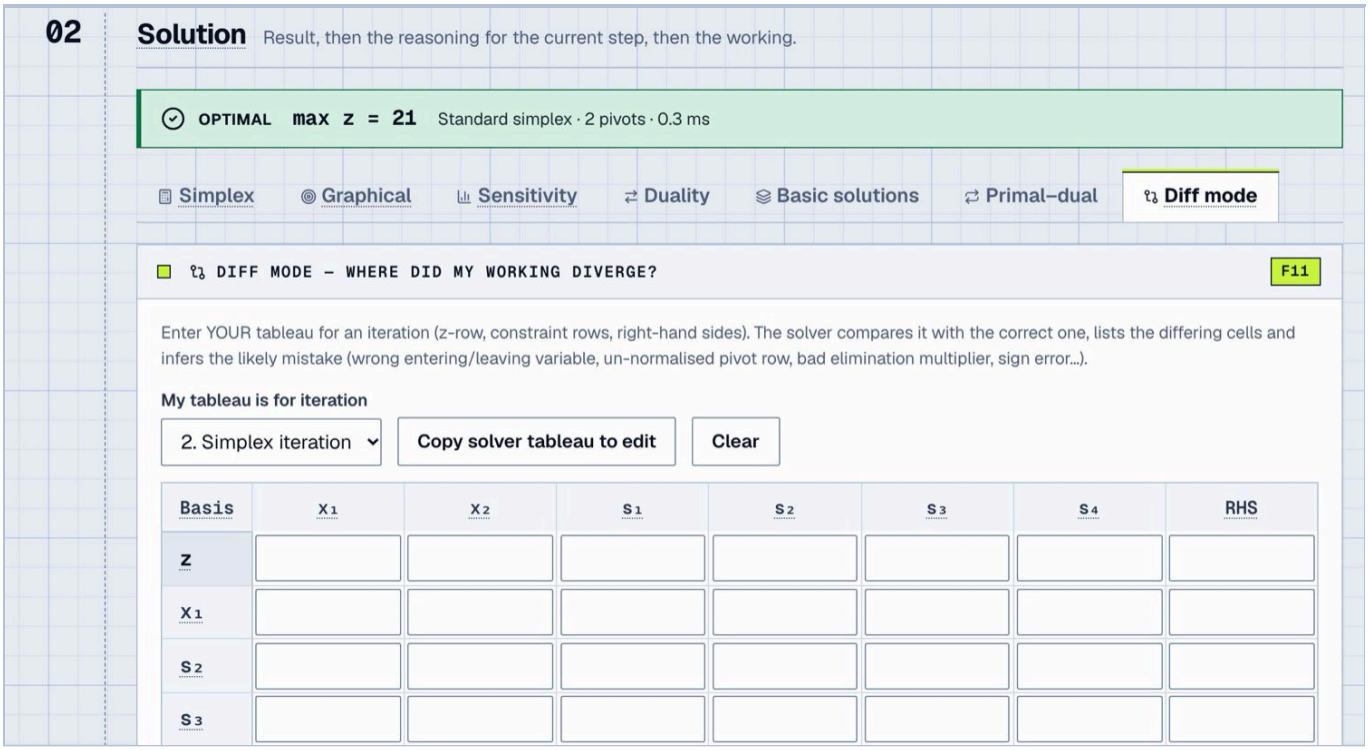


FIG 6.2 · DIFF MODE IN LINEAR PROGRAMMING: CHOOSE THE STEP, FILL YOUR TABLEAU, THEN COMPARE.

1. Open **Diff mode** and choose *My tableau is for iteration* (the step you are checking).
2. Type your numbers in the grid. **Copy solver tableau to edit** fills it with the correct one so you can change a cell on purpose to see the message.
3. Press **Compare with iteration**, or **Find the closest iteration** if you are not sure which step you did.

Diff mode is in Linear programming, Integer programming, Transportation, Assignment, Network models, Project planning and Zero-sum games.

6.4 The problem library

The library holds **66 worked problems**, one or more for every module: textbook examples, special cases (unbounded, infeasible, degenerate) and the course experiments. Open it from the left rail, or search for a problem with `/`.

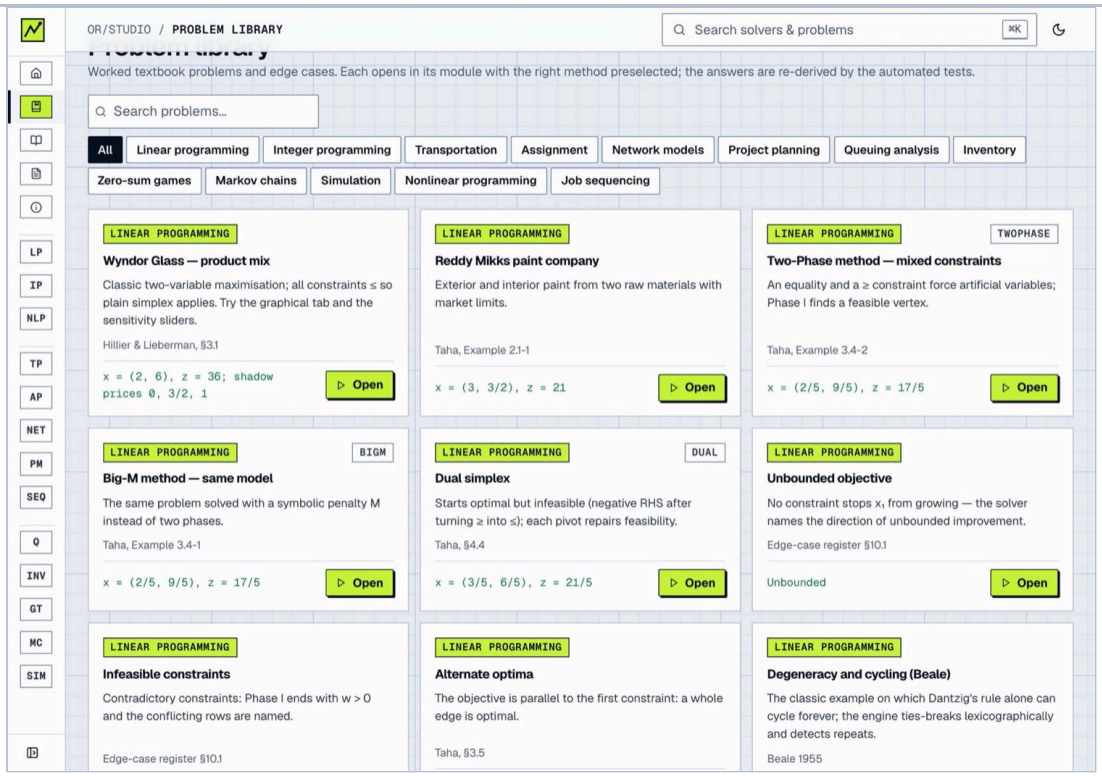


FIG 6.3 · THE LIBRARY. FILTER BY MODULE, SEARCH BY WORD, AND PRESS OPEN.

- Each card shows the source, a short description and the **expected answer** in green.
- **Open** loads the problem in its module with the right method chosen.
- Every answer is checked by the automatic tests of the app, so you can trust it.
- This guide gives the library id (such as `lp-reddy`) with each example. A link ending in `?lib=lp-reddy` opens it directly.

6.5 Presentation mode

See chapter 4.1. Press `P` to enlarge the screen for a class, and `P` or `Esc` to leave.

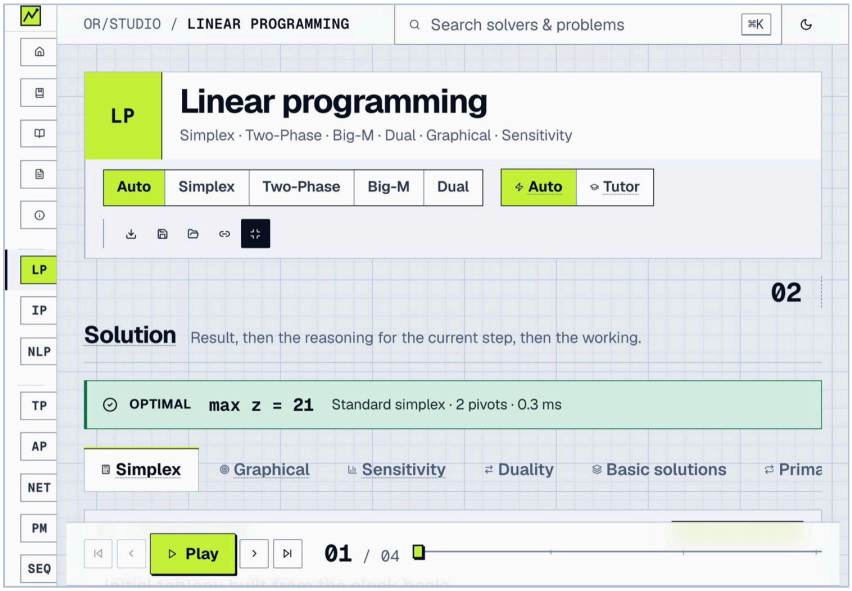


FIG 6.4 · PRESENTATION MODE: THE MODEL IS HIDDEN AND THE TEXT IS BIGGER. THE SCREEN IS NARROW HERE, SO THE BUTTONS WRAP.

07

07 / SAVE, SHARE AND EXPORT

Save, share and export

Keep your work, send it to a friend, or take the solution into a document.

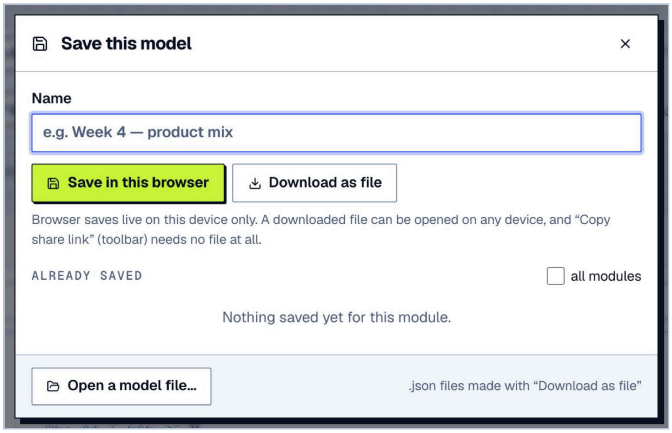
SHEET
07 OF 25
USER GUIDE

7.1 Autosave

You do not need to press anything. About half a second after you stop changing a model, the app saves it in your browser. When you return to the module, your last model is there. Each module has its own autosave. The footer of the workspace says *Autosaved in this browser*.

> **WATCH OUT**
> Autosave keeps only the **latest** model of each module. To keep several, use **Save** and give each model a name. If the browser's storage is full or blocked, the app tells you, and you can download the model as a file instead.

7.2 The Save and Open dialogs



Save is the disk icon in the toolbar. **Open** is the folder icon next to it.

1. Click **Save** and type a name, for example *Week 4 — product mix*.
2. Press **Save in this browser**. If the name already exists, the button says **Replace**.
3. Or press **Download as file** to get a `.json` file you can send or keep.

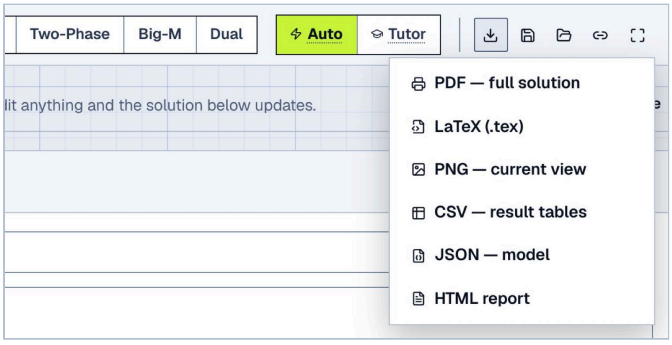
FIG 7.1 · SAVE THIS MODEL.

In **Open** you see all models saved for this module. Tick **all modules** to see every module. For each model you can open it (click its name), **rename** it (pencil), **download** it, or **delete** it (the app asks *Delete?* first). **Open a model file…** loads a `.json` file from your computer. A model made for another module opens in that module.

7.3 Share link

Click the **link icon** in the toolbar. The app copies a web address to the clipboard and tells you how long it is. Paste it into a message. Whoever opens it sees **your model, your method and your step**. No file and no account are needed. The model is stored inside the link, so very large models may give a link that is too long for some chat apps.

7.4 Export



The export menu turns the current solution into a file. Reports include the model, every step with its explanation, the result and the views you were looking at.

If nothing is solved yet, the app says *Nothing to export yet*.

FIG 7.2 · THE EXPORT MENU (DOWNLOAD ICON).

CHOICE	WHAT YOU GET	USE IT FOR
PDF — full solution	A clean report opens in a print window.	Handing in a solution or printing notes. Choose **Save as PDF** as the printer. Allow pop-ups if the browser blocks the window.
LaTeX (.tex)	A document that uses the `amsmath` and `booktabs` packages.	Theses and papers written in LaTeX.
PNG — current view	A picture of the view you are looking at.	Slides and notes: a graph, a tableau, a Gantt chart.
CSV — result tables	The result tables as comma-separated text.	Opening the numbers in Excel or other software.
JSON — model	The model only, as a small file.	Backup, or sending the problem to someone. Same as Download as file.
HTML report	The report as a web page file.	Sharing a readable solution with no software needed.

TIP

To keep a **PDF** with the step shown on screen as a picture, choose the step you want first. The report includes a snapshot of the current view.

LP

08 / LINEAR PROGRAMMING

Linear programming

Simplex · Two-Phase · Big-M · Dual · Graphical · Sensitivity · Duality · Basic solutions · Primal–dual

SHEET 08 OF 25

OPTIMISE

1 WHAT IT SOLVES

You choose how much of several things to do (the **variables**) to make one number as **big** (profit) or as **small** (cost) as possible (the **objective**). Your choices are limited by **constraints**, such as the hours or material you have. In linear programming, everything is a straight-line relation.

2 WHEN TO USE IT

- Product mix: how many of each product to make with limited machines, time or material.
- Blending and diet: the cheapest mix that meets minimum needs.
- Any "best use of limited resources" problem where doubling the input doubles the effect.

If answers must be whole numbers, use Integer programming. If the relation is curved, use Nonlinear programming.

3 HOW TO ENTER YOUR PROBLEM

1. Open **Linear programming** from the rail (code **LP**).
2. In **01 Model**, type the model (chapter 5.1), use the **Form** tab, or choose an **Example**.
3. Look for *parsed* in green under the box. If you see red text, fix the line it points to.
4. Choose a **Method** in the title block (see the table below). **Auto** is the right choice if you are not sure.
5. Use the step player to walk through the simplex tableaux.

METHOD	WHAT IT DOES
Auto	Uses the plain Simplex if every row is ≤ with a right-hand side of 0 or more. Otherwise it uses Two-Phase.
Simplex	The standard tableau method, starting from the slack variables.
Two-Phase	Phase I finds a feasible starting point with artificial variables. Phase II then optimises the real objective.
Big-M	Artificial variables with a very large penalty *M*. The app keeps M as a symbol, never as a number.
Dual	Dual simplex. It needs a start that is optimal but not yet feasible (a minimisation with costs of 0 or more). If it cannot start, the app switches method and tells you why.

4 WHAT YOU GET

The banner shows the status and the best value. The tabs are:

Simplex	Each tableau, the pivot, the ratio test, and below it the **Solution** card: variable values, slack of each constraint and its **shadow price**.
Graphical	Only for **two variables**. Draw the constraints and the feasible region and see the best corner. You can drag a constraint's right-hand side.
Sensitivity	How far each right-hand side and cost can change before the answer changes. Range bars, a what-if slider and a sentence in plain words.
Duality	The **dual** problem written out, a table that matches primal and dual parts, and checks (such as strong duality).
Basic solutions	Every possible basis, with its point and objective value. It marks the feasible ones (the corner points).
Primal–dual	The primal–dual algorithm, with its own step player.
Diff mode	Compare your hand-made tableau with the correct one (chapter 6.3).

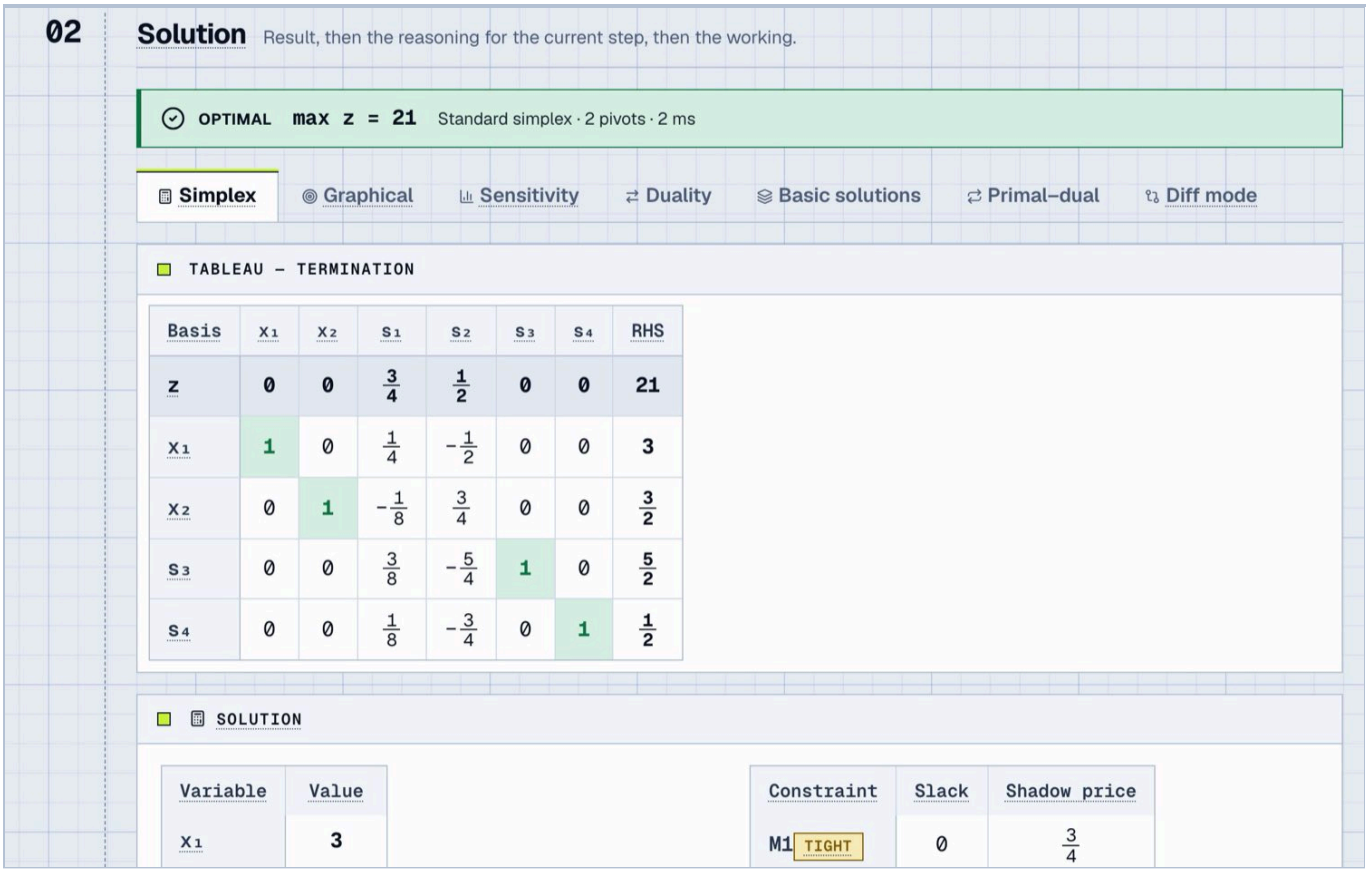


FIG 8.1 · THE LAST TABLEAU OF THE REDDY MIKKS PROBLEM. THE BOTTOM SHOWS VARIABLE VALUES AND SHADOW PRICES.

5 HOW TO READ THE RESULT

- **The z row** of the final tableau has no negative number (for a maximisation). That is the sign that the answer is optimal.
- **Basis** lists the variables that have a value. Every other variable is 0. **RHS** gives their values.
- **Slack** is the unused part of a resource. A constraint with slack 0 is `TIGHT`: it is fully used.
- **Shadow price** is how much the best value improves if you get **one more unit** of that resource. A constraint that is not tight has a shadow price of 0.
- On the **Sensitivity** tab, the green bar is the range where the shadow price stays valid.
- On the **Duality** tab, *All checks hold* means primal and dual agree.

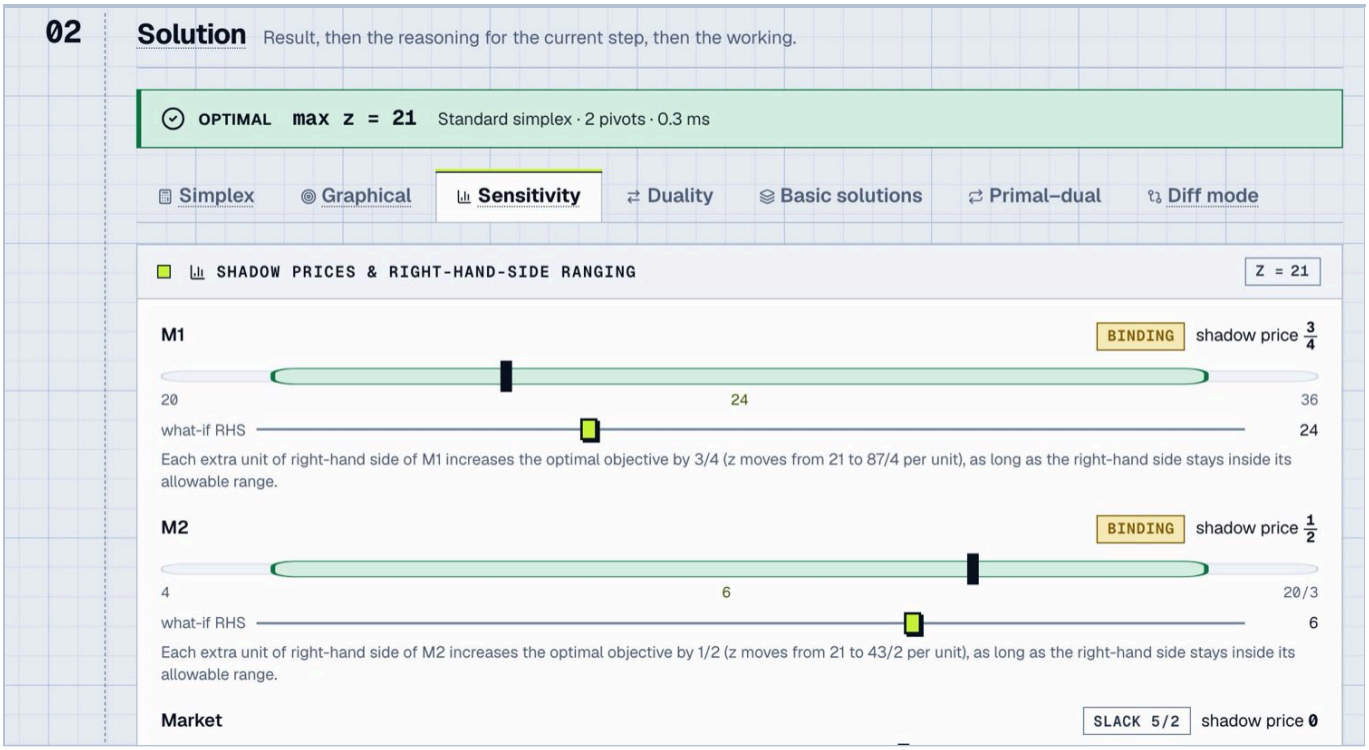


FIG 8.2 · SENSITIVITY. THE M1 CONSTRAINT (24) CAN MOVE BETWEEN 20 AND 36 AND THE SHADOW PRICE ¾ STAYS VALID.

6 WORKED MINI EXAMPLE

Library problem `lp-reddy` (Reddy Mikks paint company, Taha 2.1-1). Exterior paint (x_1) earns 5 and interior paint (x_2) earns 4 per unit.

```
max 5x1 + 4x2
subject to
  M1: 6x1 + 4x2 <= 24
  M2: x1 + 2x2 <= 6
  Market: -x1 + x2 <= 1
  Demand: x2 <= 2
```

Result	OPTIMAL `x₁ = 3`, `x₂ = 3/2`, `max z = 21`, after 2 pivots (4 steps).
Shadow prices	M1 = **3/4**, M2 = **1/2**, Market = 0, Demand = 0. M1 and M2 are tight (slack 0). Market has slack 5/2 and Demand has slack 1/2.
Basic solutions tab	The standard form has 6 variables and 4 equations, so there are C(6, 4) = 15 candidate bases. **6** are feasible corner points, **8** are infeasible and **1** is not a basis.
Duality tab	The dual is `min 24y₁ + 6y₂ + y₃ + 2y₄`. Its answer is also 21.

A second example, `lp-twophase`, minimises `4x1 + x2` with an equality and a ≥ row. Auto chooses Two-Phase. The answer is `x = (2/5, 9/5)` and `z = 17/5`. Walk-through A follows Reddy Mikks from the first key press.

7 TIPS AND COMMON MISTAKES

WATCH OUT

- Check the **direction** of each sign (≤ or ≥). Reversing one changes the whole problem.
- A **negative right-hand side** or a ≥ row needs artificial variables. That is normal. Auto switches to Two-Phase for you.
- The **Graphical** tab disappears when the model has more than two variables. This is not an error.
- An answer of *Infeasible* or *Unbounded* usually means a typing mistake. See chapter 21.

TIP

Name your rows (`labour: …`). The shadow price table then shows *labour* instead of (1), which is much easier to read.

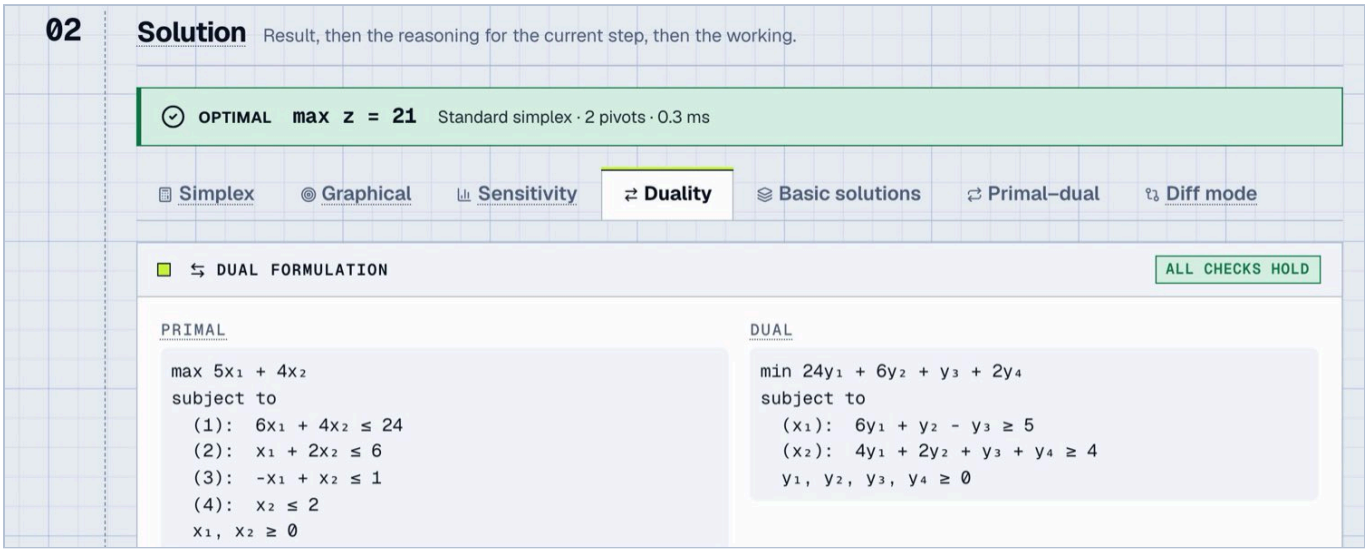


FIG 8.3 · DUALITY TAB: THE PRIMAL AND ITS DUAL SIDE BY SIDE.

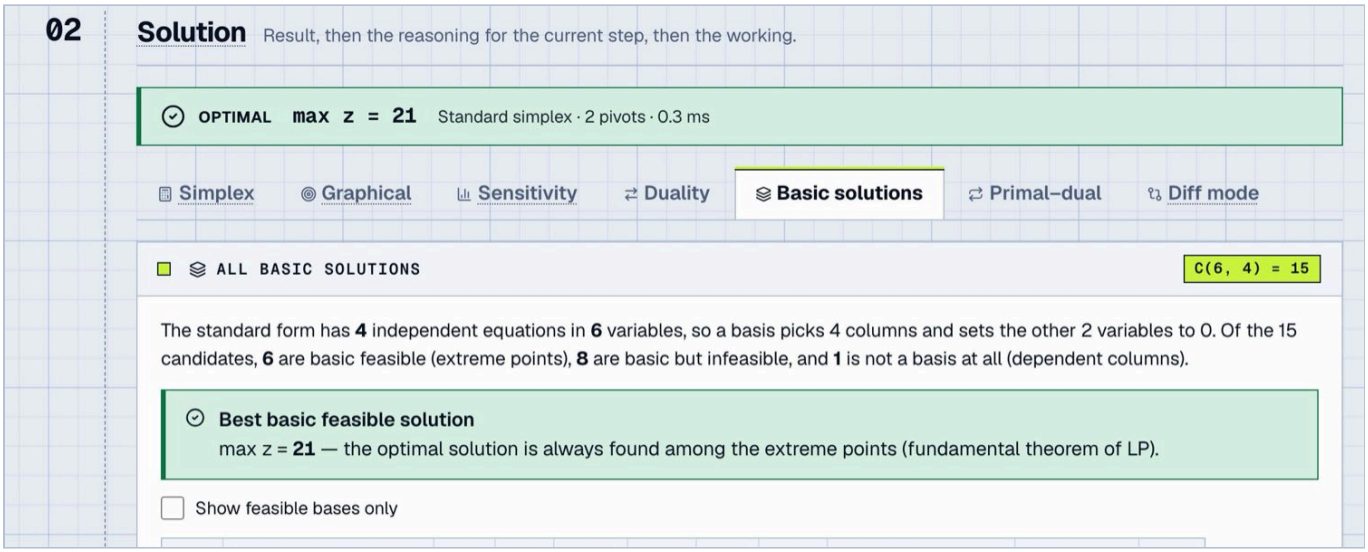


FIG 8.4 · BASIC SOLUTIONS: 15 CANDIDATE BASES, OF WHICH 6 ARE FEASIBLE CORNER POINTS (BFS).

IP

09 / INTEGER PROGRAMMING

Integer programming

Branch and bound with a tree · Gomory cuts

SHEET 09 OF 25

OPTIMISE

1 WHAT IT SOLVES

It is linear programming with one extra rule: some or all variables must be **whole numbers** (**integers**), or **0 or 1** (**binary**). You cannot build 2.5 machines.

2 WHEN TO USE IT

- Counting things: machines, vehicles, workers.
- Yes/no choices: build or not, take an item or not (the knapsack problem).
- **Mixed** problems where only some variables must be whole.

3 HOW TO ENTER YOUR PROBLEM

1. Open **Integer programming**. Type the model like an LP and add a declaration line: `int x1, x2` or `bin x1, x2`. The Form tab has the same choices in its **Type** row.
2. Choose the **Method**: **Branch & bound** or **Gomory cuts**.
3. For branch and bound, open **Search settings**: *Node selection* (Best bound first, Depth first or Breadth first), *Branching variable* (First fractional or Most fractional) and a *Node limit*. The defaults are fine to begin with.
4. Step through the solution with the player.

4 WHAT YOU GET

- **Branch and bound:** a **Tree** tab (drag to pan, scroll to zoom, click a node for details), an *incumbent and bound* chart, a **Plane** tab for two variables, and Diff mode.
- **Gomory cuts:** a **Cuts** tab with the tableau and the list of cuts, each written in the original variables, and a Plane tab that draws the cuts.

5 HOW TO READ THE RESULT

- **Branch and bound** solves the LP, finds a variable that is not whole (say 1.5) and splits the problem into two: one with `x ≤ 1`, one with `x ≥ 2`. It repeats. Each box in the tree is one LP.
- Node symbols: ◆ split further, ✓ all whole, ★ best whole answer so far (the *incumbent*), ✂ cut off because it cannot beat the incumbent, ⊘ no solution.
- The **bound** is the best value any open node could still reach. When the incumbent equals the bound, the answer is proven best.
- The banner shows the best value, the LP bound, the number of nodes and how many were pruned.

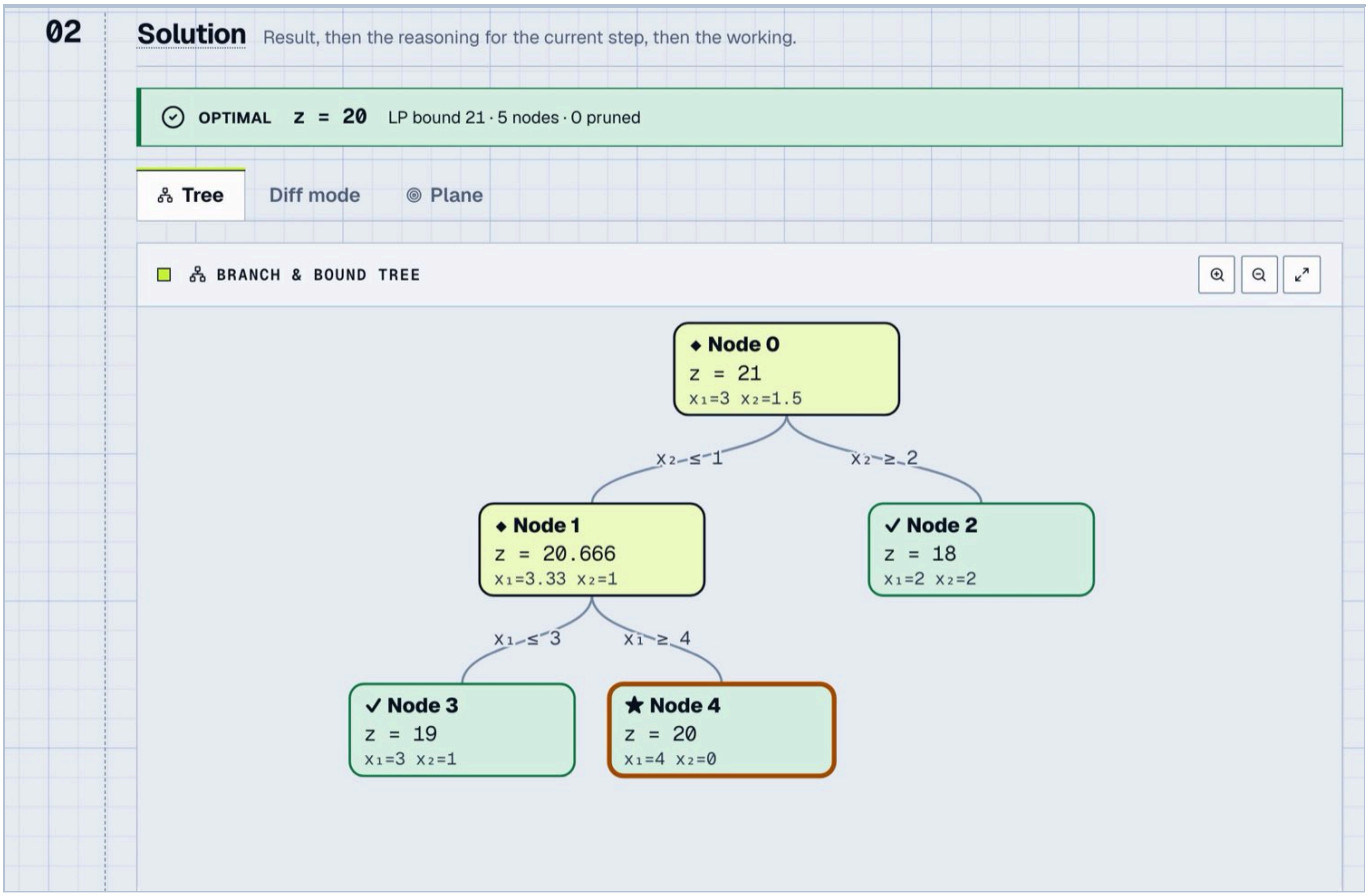


FIG 9.1 · THE FINISHED TREE FOR `INT-TAHA`. THE STAR MARKS THE BEST WHOLE ANSWER, Z = 20.

6 WORKED MINI EXAMPLE

Library problem `int-taha` (Taha 9.2-1): `max 5x1 + 4x2` subject to `6x1 + 4x2 <= 24`, `x1 + 2x2 <= 6`, `int x1, x2`.

Step 1 (root)	Ignore the whole-number rule. The LP gives $x_1 = 3$, $x_2 = 3/2$, $z = 21$. This is an upper bound.
Branching	$x_2 = 1.5$ is not whole, so split into $x_2 \le 1$ and $x_2 \ge 2$. Then split $x_1 \le 3$ and $x_1 \ge 4$.
Result	`OPTIMAL` $x = (4, 0)$, $z = 20$, found with **5 nodes**. The gap to the LP bound (21) is 1.
Gomory method	The same problem needs **4 cuts** and reaches the same $z = 20$.

7 TIPS AND COMMON MISTAKES

WATCH OUT

- **Do not round** the LP answer. Rounding (3, 1.5) up to (3, 2) breaks a constraint. That is why branch and bound exists.
- If the search stops with a warning, you have reached the **node limit**. Raise it, or simplify the model.
- A problem can be feasible as an LP but have **no whole-number answer**. Try `int-infeasible` : `2x1 = 3` with `int x1`.

TIP

Use `int-knap` (a 0-1 knapsack, best value 28) to see a binary problem. Try the three node selections and compare the trees.

NLP

10 / NONLINEAR PROGRAMMING

Nonlinear programming

Gradient and Newton steps · KKT check · exact quadratic programming

SHEET 10 OF 25

OPTIMISE

1 WHAT IT SOLVES

It finds the lowest (or highest) value of a **curved** function, such as `(x1 - 3)² + 2(x2 + 1)²`, with or without constraints. It gives three tools, chosen with the **Method** switch.

2 WHEN TO USE IT

- **Unconstrained:** find the minimum of a function with an iterative method.
- **KKT:** check whether a point satisfies the Karush–Kuhn–Tucker conditions (the standard test for a constrained optimum).
- **Quadratic:** solve *minimise* $\frac{1}{2}x^TQx + c^Tx$ *subject to* $Ax \le b$, $x \ge 0$ exactly.

3 HOW TO ENTER YOUR PROBLEM

1. Open **Nonlinear programming** and choose **Unconstrained**, **KKT** or **Quadratic**.
2. **Unconstrained:** type **f(x_1, x_2, …)** using `x1, x2`, the signs `+ - * / ^` and the functions `exp, ln, sin, cos, sqrt`. Type a **Starting point** such as `0, 0`. Pick **Newton** or **Gradient descent**.
3. **KKT:** type the objective, add constraints with **+ constraint** (each is `g(x) ≤ 0` or `h(x) = 0`), and type the **Candidate point**.
4. **Quadratic:** fill the symmetric matrix **Q**, the vector **c**, and the constraint grid **A | b**.

4 WHAT YOU GET

- **Unconstrained:** a **contour plot** with the path of the iterates, symbolic derivatives (the gradient and the Hessian), and a step player.
- **KKT:** a list of the four conditions, each marked satisfied or not, with the multipliers λ.
- **Quadratic:** the exact optimum, and whether the problem is convex.

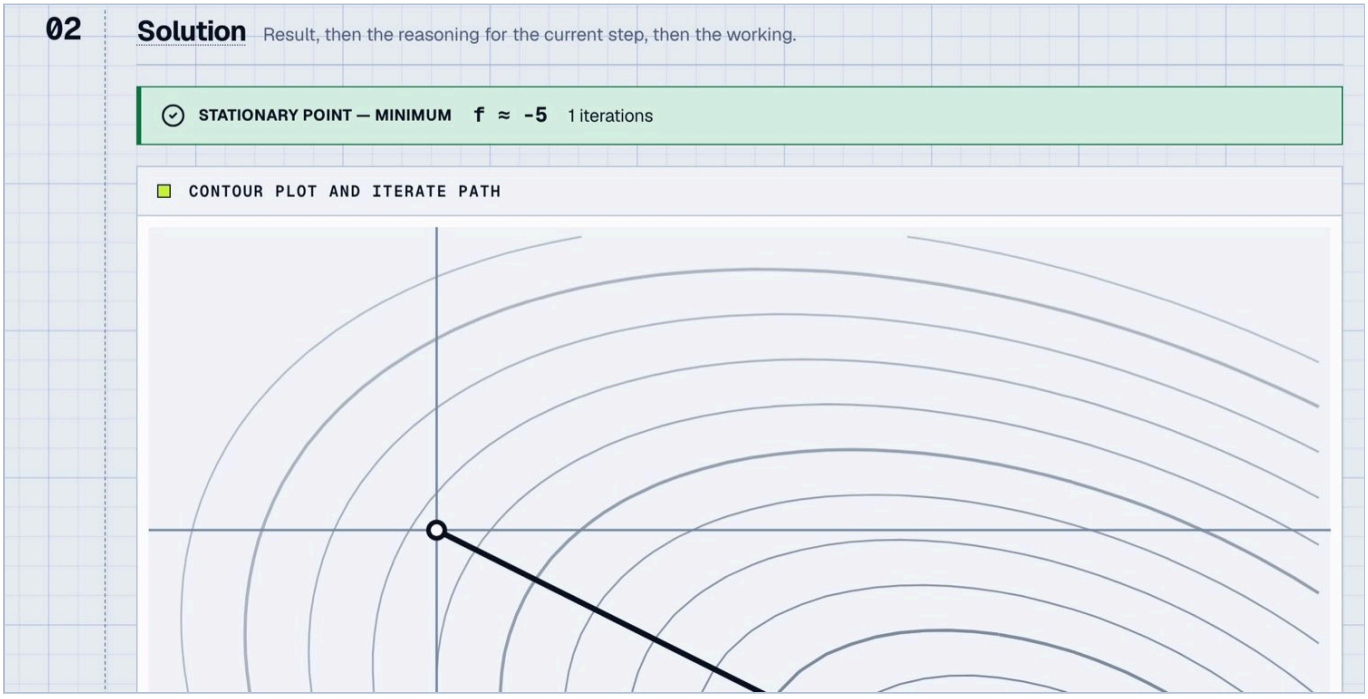


FIG 10.1 · THE CONTOUR PLOT OF `NLP-BOWL`. NEWTON'S METHOD REACHES THE MINIMUM IN ONE STEP.

5 HOW TO READ THE RESULT

- **Stationary point — minimum:** the gradient is almost zero and the Hessian eigenvalues are all positive. It is a local minimum, and the global one if the function is convex.
- **Contours** are lines of equal f. The path should move across them towards the centre.
- **KKT satisfied** means the point is a *candidate* optimum. It is sufficient only if the problem is convex.

6 WORKED MINI EXAMPLE

`nlp-bowl`	`f = (x1 - 3)^2 + 2*(x2 + 1)^2 + x1*x2`, start `(0, 0)`, Newton. Result: stationary point **(4, −2)**, **minimum**, `f = -5`, after **1 iteration**. Gradient descent needs more steps and zig-zags.
`nlp-kkt`	Is `(1, 1)` a KKT point of `min (x₁−2)² + (x₂−2)²` subject to `x₁ + x₂ − 2 ≤ 0`? `KKT SATISFIED`, with multiplier **λ = 2** on the active constraint.
`nlp-qp`	Quadratic program with `Q = [[2,0],[0,2]]`, `c = (-4, -4)`, `x₁ + x₂ ≤ 2`. Exact optimum **x = (1, 1)**, objective **−6**. The problem is convex, so this is the global minimum.

7 TIPS AND COMMON MISTAKES

WATCH OUT

- Write the product with a star: `2*x1`. Use `^` for powers: `x1^2`.
- Methods that follow the slope find a **local** minimum. A curve with several valleys may need several start points.
- Write constraints so that the right side is 0, in the form `g(x) ≤ 0`. For `x1 + x2 ≤ 2`, type `x1 + x2 - 2`.

TP

11 / TRANSPORTATION

Transportation

Starting solution · MODI optimality test · closed loops · method comparison

SHEET
11 OF 25
FLOWS AND PLANS

1 WHAT IT SOLVES

Goods are in stock at several **sources** (factories, warehouses). Several **destinations** (shops, cities) need them. Shipping one unit on each route has a cost. The module finds how many units to send on each route so that the **total cost is lowest**.

2 WHEN TO USE IT

- Moving goods, fuel, water or people between places.
- Matching supply to demand where only the unit costs differ.

3 HOW TO ENTER YOUR PROBLEM

1. Open **Transportation**. In the grid, type the **unit cost** of each route (rows are sources, columns are destinations).
2. Type the **Supply** of each source (right column) and the **Demand** of each destination (bottom row).
3. Add or remove rows and columns with the **+** and **–** buttons. Rename sources and destinations.
4. Choose **Minimise cost** or **Maximise profit**. Type `M` or `x` in a cell for a route that cannot be used.
5. Choose the starting method in the title block: **Vogel** (VAM), **Least-cost** or **NW corner**.

If supply and demand are not equal, the app adds a labelled **dummy** row or column by itself and tells you.

4 WHAT YOU GET

- **Allocation:** the grid, step by step. First the starting solution, then each **MODI** step.
- **Method comparison:** NW corner, least cost and Vogel side by side (starting cost and number of improvement steps).
- **Diff mode** to check your own allocation.

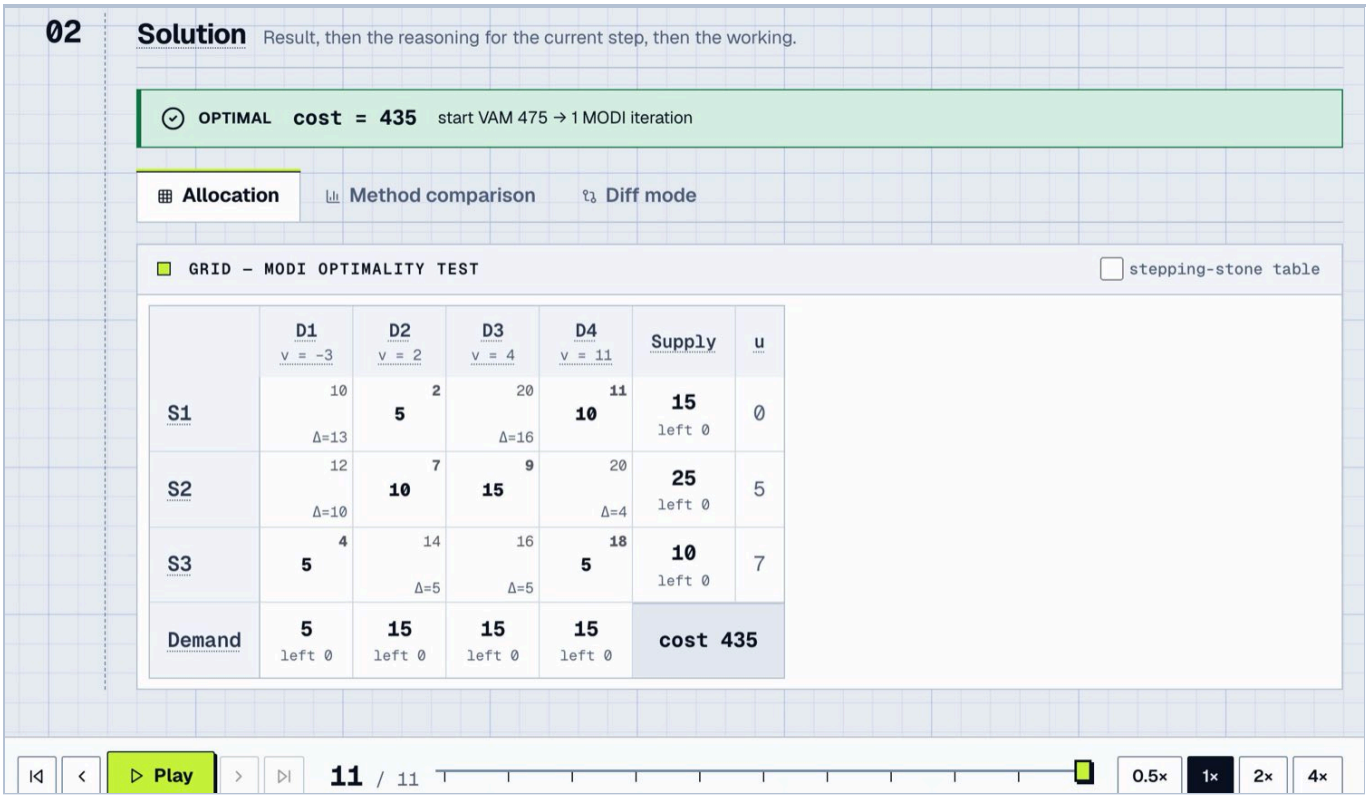


FIG 11.1 · THE LAST STEP OF `TR-TAHA`. EACH CELL SHOWS THE UNIT COST (SMALL) AND THE UNITS SENT (LARGE), WITH U, V AND Δ.

5 HOW TO READ THE RESULT

- A cell with a big number is a **route in use**. The small number is its cost.
- **u** and **v** are *dual values* for each row and column. MODI finds them so that every used cell has `cost = u + v`.
- **Δ** (improvement index) of an empty cell is `cost − u − v`. If **every Δ ≥ 0**, the solution is optimal. A negative Δ shows a cheaper route, and the app draws the **closed loop** used to move units.
- A diagnostic *degenerate* means a used cell has 0 units. It is normal; the app places a zero (ε) so the method can go on.

6 WORKED MINI EXAMPLE

Library problem `tr-taha` (SunRay Transport, Taha 5.1-1). Supply `15, 25, 10`; demand `5, 15, 15, 15`.

	D1	D2	D3	D4
S1	10	2	20	11
S2	12	7	9	20
S3	4	14	16	18

With **Vogel**, the starting cost is **475**. One MODI step brings it to `OPTIMAL` **435**. The shipments are S1→D2: 5, S1→D4: 10, S2→D2: 10, S2→D3: 15, S3→D1: 5 and S3→D4: 5. The dual values are u = (0, 5, 7) and v = (−3, 2, 4, 11). The **Method comparison** tab shows how the other starts compare.

7 TIPS AND COMMON MISTAKES

WATCH OUT

- For **profits**, choose *Maximise profit*. Do not type negative costs.
- The numbers on the grid must be **per unit**, not totals for a route.
- Different starting methods can give different starting costs, but the **final optimal cost is the same**.

AP

12 / ASSIGNMENT

Assignment

Hungarian method · exact minimum line cover · rectangular · maximisation

SHEET 12 OF 25

FLOWS AND PLANS

1 WHAT IT SOLVES

It matches **each worker to exactly one job** so that the total cost (or time) is as low as possible. The cost of every worker–job pair is in a table.

2 WHEN TO USE IT

- Workers to jobs, machines to tasks, teachers to classes, drivers to routes.
- When profits matter more than costs, use **Maximise profit**.

3 HOW TO ENTER YOUR PROBLEM

1. Open **Assignment**. Type the cost of each worker (row) doing each job (column).
2. Add or remove rows and columns with **+** and **–**. Rename them.
3. Choose **Minimise cost** or **Maximise profit**. Type `M` or `x` in a cell if a worker cannot do a job.
4. Step through the **Hungarian** tab. A table that is not square is padded with a dummy row or column of zeros.

4 WHAT YOU GET

The **Hungarian** tab shows the matrix at each stage, the lines that cover the zeros, and finally the **optimal assignment** with the cost of each pair. Tutor mode and Diff mode are available.

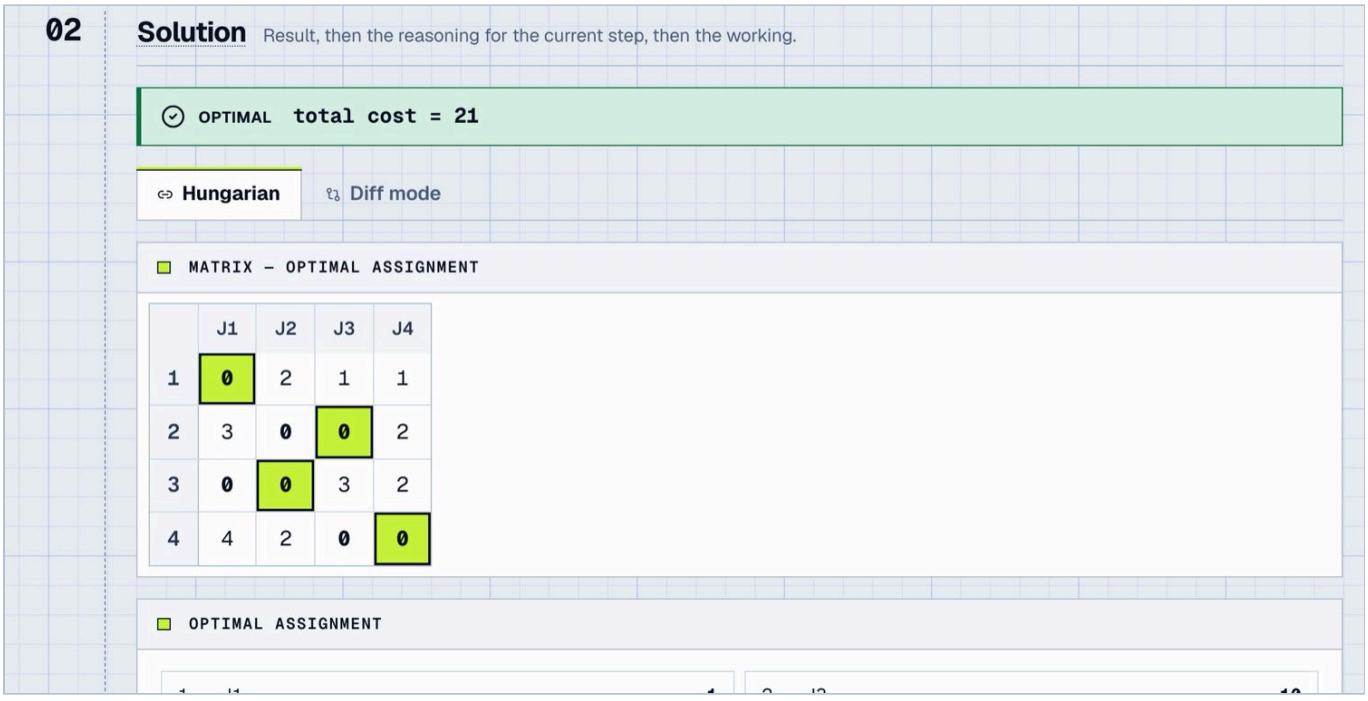


FIG 12.1 · THE FINAL STAGE FOR `AS-4X4`: THE REDUCED MATRIX WITH ITS INDEPENDENT ZEROS, AND THE LIST OF PAIRS.

5 HOW TO READ THE RESULT

- The method **subtracts the smallest number** of each row and then of each column. This makes zeros without changing the best answer.
- It then covers all zeros with as **few lines** as possible. If the number of lines equals the size of the table (n), n independent zeros give the assignment. If fewer, the table is **adjusted** and the loop repeats.
- The **total cost** is always taken from the **original** table, not the reduced one.

6 WORKED MINI EXAMPLE

Library problem `as-4x4` : four workers and four jobs. Costs: `1 4 6 3 / 9 7 10 9 / 4 5 11 7 / 8 7 8 5` .

Result	OPTIMAL **total cost = 21**, in 7 steps. The first line cover uses fewer than 4 lines, so the matrix is adjusted once.
Assignment	1 → J1 (cost 1), 2 → J3 (10), 3 → J2 (5), 4 → J4 (5).

Other library problems: `as-rect` (2 workers, 3 jobs, so a dummy worker is added; total 9), `as-max` (maximise profit) and `as-3x3` (total 9).

7 TIPS AND COMMON MISTAKES

WATCH OUT

- Use *Maximise profit* for a table of profits or scores. Do not just reverse the sign yourself.
- A dummy worker or job in a rectangular table has no real cost. In the answer, a worker paired with a dummy job simply gets **no job**.

TIP

Use Tutor mode to practise the two decisions of the method: how to cover all zeros with the fewest lines, and which zeros to choose.

NET

13 / NETWORK MODELS

Network models

Draw the graph · shortest paths · spanning trees · maximum flow

SHEET 13 OF 25

FLOWS AND PLANS

1 WHAT IT SOLVES

A **network** has **nodes** (places) joined by **edges** (roads, pipes, cables). This module answers three kinds of question: the **shortest route**, the **cheapest way to connect everything** (spanning tree), and the **largest flow** from a source to a sink.

2 WHEN TO USE IT

- Routes in a road or data network, cable layouts, water or traffic flow.
- Finding the weakest link: the **minimum cut** equals the maximum flow.

3 HOW TO ENTER YOUR PROBLEM

1. Open **Network models**. Draw the graph with the tools (chapter 5.4): **Add node**, **Add edge**, **Move**, **Delete**. Or edit the table of edges.
2. Give each edge a **Weight** (length or cost). For max flow, give a **Cap.** (capacity). Tick **directed** for a one-way edge.
3. Choose the **Source** and the **Sink / target**.
4. Choose the method: **Dijkstra**, **Bellman–Ford**, **Floyd** (all pairs), **Kruskal**, **Prim** (spanning tree) or **Max flow**.

4 WHAT YOU GET

A **Graph** tab that is redrawn at each step, with labels on the nodes. For shortest paths, it also gives a table of distances; for max flow, a table of flow on each edge and the minimum cut. There is also a Diff mode and a Tutor mode.

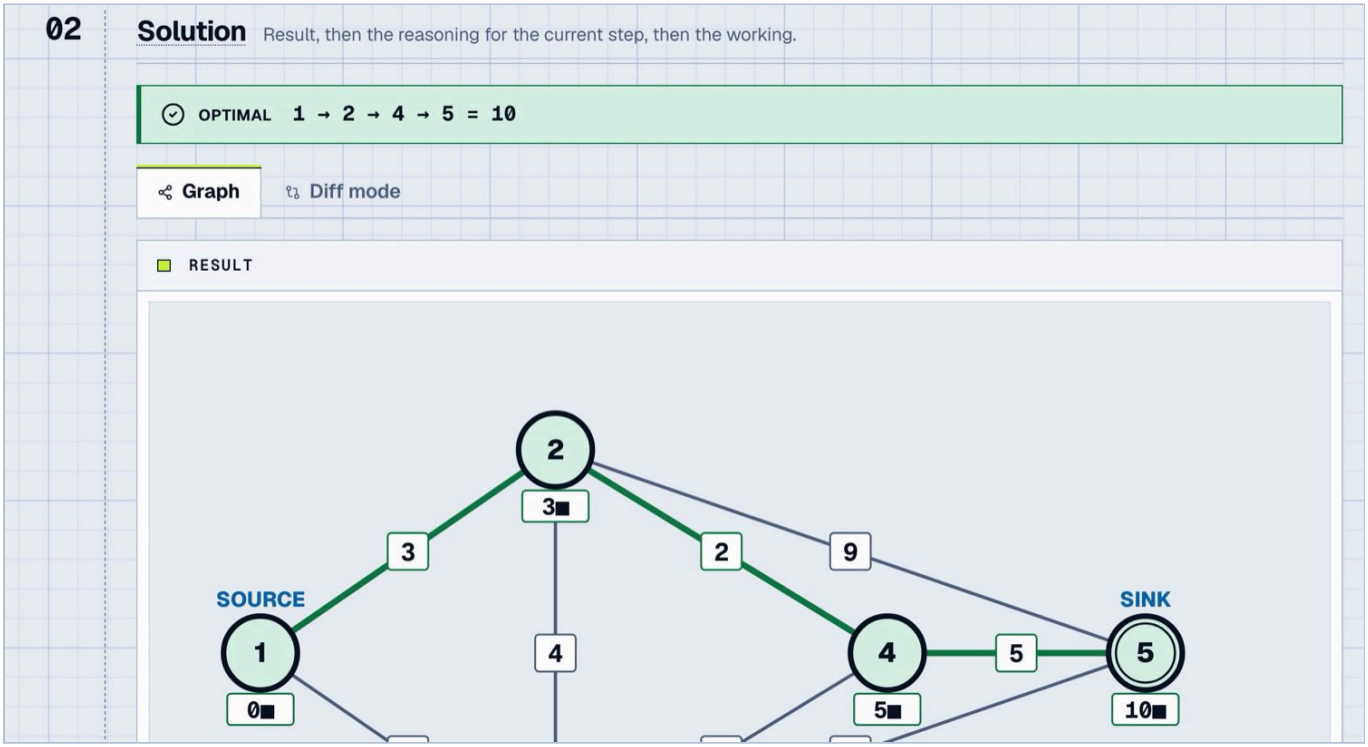


FIG 13.1 · THE RESULT FOR `NET-SHORT`: THE SHORTEST ROUTE FROM NODE 1 TO NODE 5 AND THE DISTANCE TO EVERY NODE.

5 HOW TO READ THE RESULT

- **Dijkstra** gives each node a distance label. A node becomes *permanent* when its label cannot improve. The **Via** column tells the node you came from, so you can follow the route backwards.
- **Spanning tree:** the chosen edges join all nodes with the least total weight and no loop.

Max flow shows `flow/capacity` on each edge. **Saturated** edges are full. The **min cut** edges are the bottleneck.

6 WORKED MINI EXAMPLE

`net-short`	Dijkstra, 13 steps. Shortest route **1 → 2 → 4 → 5, length 10**. Distances: node 2 = 3, node 3 = 7, node 4 = 5, node 5 = 10.
`net-flow`	Max flow from S to T: **15**. Edges S→A (10/10) and D→T (10/10) are saturated. The minimum cut is made of A→C and D→T, with capacity 5 + 10 = 15, equal to the flow.
`net-neg`	A graph with a negative edge. Dijkstra refuses it; **Bellman–Ford** handles it.

7 TIPS AND COMMON MISTAKES

WATCH OUT

- **Dijkstra cannot use negative weights.** Switch to Bellman–Ford.
- Make sure **Source** and **Sink** are set (they show a dash when empty).
- An edge is **two-way** unless you tick *directed*. For a one-way road, tick it.

TIP

You can draw the graph in any layout. Moving a node only changes the picture, never the answer.

PM

14 / PROJECT PLANNING

Project planning

CPM · PERT · floats · Gantt chart · network diagram · time–cost crashing

SHEET 14 OF 25

FLOWS AND PLANS

1 WHAT IT SOLVES

A project is a set of **activities**, and some cannot start until others finish. This module finds the **shortest time** to finish the whole project, and the **critical path**: the chain of activities that cannot be delayed without delaying the project.

2 WHEN TO USE IT

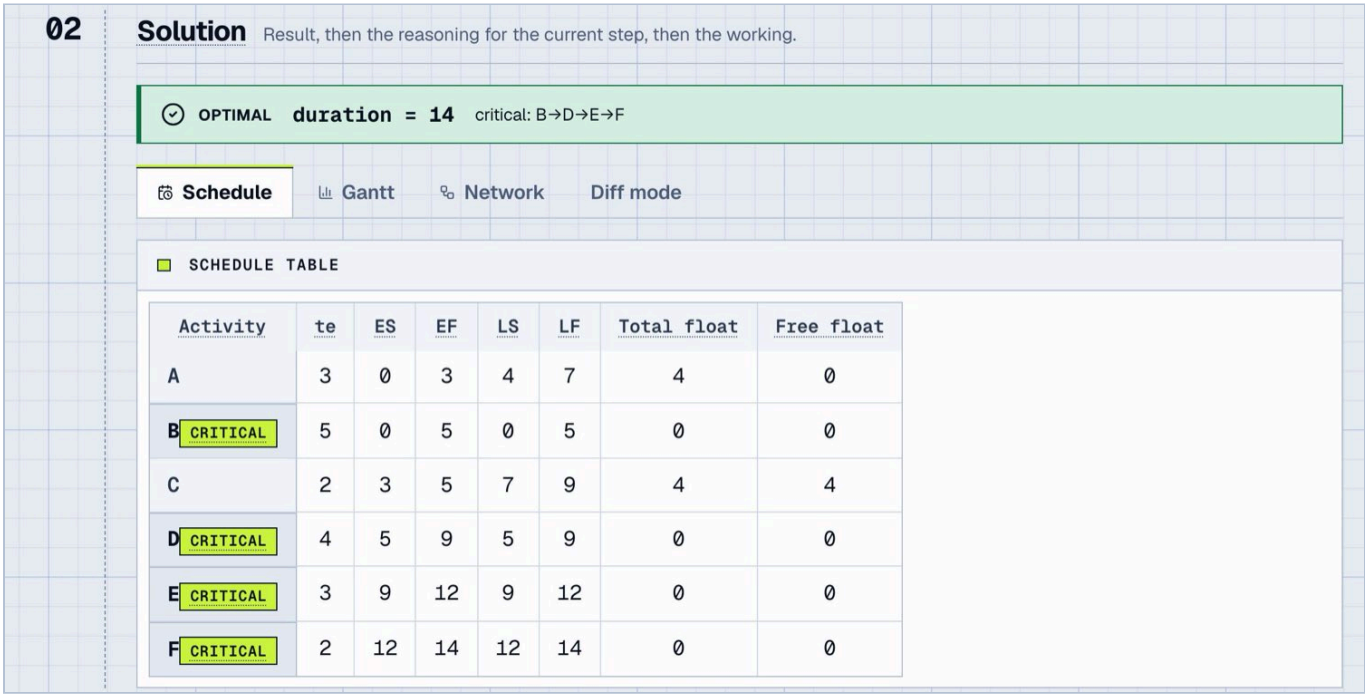
- **CPM** when the time of each activity is known.
- **PERT** when times are uncertain, so you give three estimates.
- **Crashing** when you want to finish earlier by paying extra, at the least cost.

3 HOW TO ENTER YOUR PROBLEM

1. Open **Project planning** and choose **CPM**, **PERT** or **Crashing**.
2. Add a row for every activity. **Id** is its name (A, B…). **After** lists the activities that must finish first, separated by commas (for example `C, D`). Leave it empty if it can start at once.
3. CPM: type the **Time**. PERT: type **a** (best case), **m** (most likely) and **b** (worst case). Crashing: type the normal **Time**, **Cn** (normal cost), **Dc** (crash duration) and **Cc** (crash cost).
4. Optionally type a **Deadline**. For PERT you can choose a **Target time T** in the result.

4 WHAT YOU GET

- **Schedule:** the table te, ES, EF, LS, LF and floats for each activity, built step by step (forward pass, then backward pass).
- **Gantt:** bars on a time line. **Network:** the activity-on-node diagram. **Crashing:** a cost versus time chart and a plan. Diff mode and Tutor mode also exist.



Activity	te	ES	EF	LS	LF	Total float	Free float
A	3	0	3	4	7	4	0
B CRITICAL	5	0	5	0	5	0	0
C	2	3	5	7	9	4	4
D CRITICAL	4	5	9	5	9	0	0
E CRITICAL	3	9	12	9	12	0	0
F CRITICAL	2	12	14	12	14	0	0

FIG 14.1 · THE SCHEDULE OF `PR-CPM`. ROWS MARKED CRITICAL HAVE ZERO TOTAL FLOAT.

5 HOW TO READ THE RESULT

- **ES / EF** are the earliest start and finish. **LS / LF** are the latest start and finish that do not delay the project.

- **Total float** = how long an activity can be late without delaying the project. **Free float** = how long it can be late without delaying the next activity.
- **Critical** activities have **zero float**. Together they form the critical path.
- **PERT:** expected time `(a + 4m + b) / 6`; variance `((b − a) / 6)²`. The module adds the variances along the critical path and gives the **probability** of finishing by a target time.

6 WORKED MINI EXAMPLE

`pr-cpm`	A=3, B=5, C=2 (after A), D=4 (after B), E=3 (after C, D), F=2 (after E). Project duration **14**. Critical path **B → D → E → F**. A and C have 4 days of float.
`pr-pert`	Duration **14**, critical path A → C → E, variance 2.3333 (σ = 1.528). The chance to finish by **T = 16 is 90.5 %**. It is 95 % sure to finish within 16.51.
`pr-crash`	Normal duration 9 at cost 360. Crash plan: 8 days for 30 more, 7 days for 70 more, 6 days for 130 more. The project **cannot go below 6** days.

7 TIPS AND COMMON MISTAKES

WATCH OUT

- Write predecessors exactly as the activity ids, and separate them with commas.
- A **loop** (A after B, B after A) is impossible. The app reports a *Cyclic dependency* and names the loop.
- Do not mix units: use the same time unit (days or weeks) for every activity.

Q

15 / QUEUING ANALYSIS

Queuing analysis

Live λ / μ / c sliders · performance measures · cost-optimal number of servers

SHEET 15 OF 25

UNCERTAINTY

1 WHAT IT SOLVES

It studies **waiting lines**. Customers arrive at random, wait if all servers are busy, and are served. The module tells you how long people wait, how many are in line, and how busy the servers are, using formulas.

2 WHEN TO USE IT

- Bank counters, ticket windows, call centres, hospital desks, a repair shop.
- Deciding **how many servers** to hire: more servers cost money, but fewer cause long waits.

3 HOW TO ENTER YOUR PROBLEM

1. Open **Queuing analysis** and choose the model in the **Method** switch (see the table). The name is written as *arrivals / service / servers*. The letter **M** means random times with an average (a Poisson stream of arrivals, exponential service).
2. Type the **Arrival rate λ** (customers per time unit) and the **Service rate μ** (customers one server can serve per time unit).
3. Type the other numbers the model needs: servers **c**, system capacity **N**, or, for M/G/1, the spread **σ** of the service time.
4. Optional: the **cost per server C_s** and the **waiting cost Cw** (per customer per time unit) for the cost model.
5. Use the **live sliders** for λ, μ and c. The answers change as you move them.

MODEL	USE IT FOR
M/M/1	One server, unlimited waiting room.
M/M/c	c servers sharing one line (a call centre).
M/M/1/N	One server, and at most N customers in the system. Customers who find it full are lost.
M/M/c/N	c servers, at most N customers.
Machines	A small group of machines that break down and are fixed by c repairers.
M/G/1	One server, service time of any shape. You give its standard deviation σ (0 means a fixed service time).

4 WHAT YOU GET

- **Measures:** the main numbers, utilisation and probabilities, a plain-words paragraph and the formulas used.
- **Sensitivity:** a chart of the queue length against the arrival rate λ. The curve climbs steeply as ρ gets close to 1, and a dot marks your current λ.
- **Cost model:** total cost for each number of servers, and the best number.

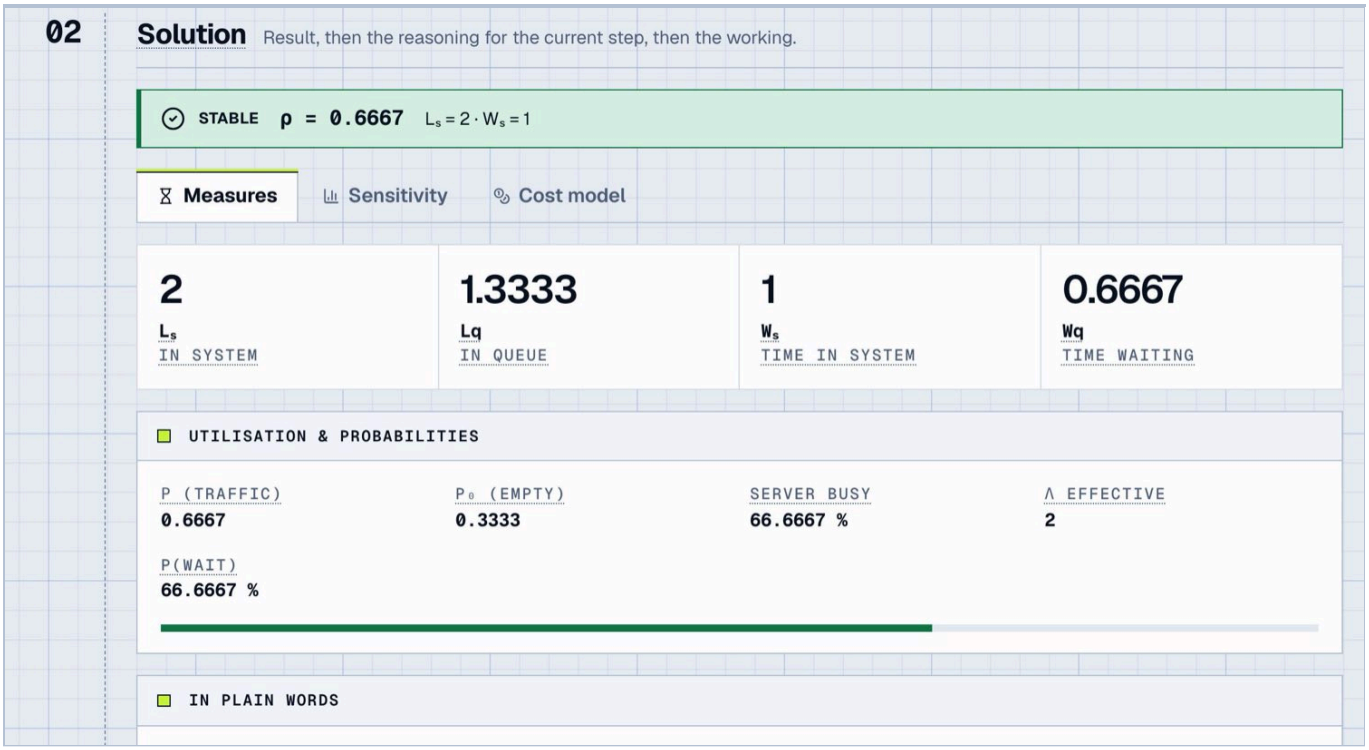


FIG 15.1 · MEASURES FOR `Q-MM1`.

5 HOW TO READ THE RESULT

ρ (rho)	Traffic intensity: how busy the servers are, `λ / (c·μ)`. It must be **less than 1**, or the queue grows forever. The banner says `STABLE` or not.
L_s, Lq	Average number of customers **in the system**, and **in the queue** only.
W_s, Wq	Average time **in the system**, and **waiting** in the queue only.
P_0, P(wait)	Chance that the system is empty, and the chance that a customer has to wait.

6 WORKED MINI EXAMPLE

Library problem `q-mm1` (one clerk): λ = 2 customers per hour, μ = 3 per hour. The result is `STABLE` with **ρ = 0.6667**, **L_s = 2**, **Lq = 1.3333**, **W_s = 1** hour and **Wq = 0.6667** hour. The clerk is busy 66.67 % of the time and 66.67 % of customers have to wait.

Library problem `q-cost`: λ = 10, μ = 3, server cost 20 per hour, waiting cost 30 per customer-hour. With 4 servers (ρ = 0.8333) the total cost is 278.66 (server cost 80 plus waiting cost 198.66). The **Cost model** tab shows the best choice is **5 servers**.

7 TIPS AND COMMON MISTAKES

WATCH OUT

- λ and μ must use the **same time unit**. Do not mix "per hour" and "per minute".
- If **ρ ≥ 1** in a model without a capacity limit, there is no steady state. Add servers or lower λ.
- μ is the rate of **one** server, not of the whole group.

TIP

Slide λ slowly towards μ and watch the queue length explode. This shows why a server that is "almost full" gives very long waits.

INV

16 / INVENTORY

Inventory

EOQ · production lots · planned shortages · quantity discounts · newsvendor

SHEET 16 OF 25

UNCERTAINTY

1 WHAT IT SOLVES

How much should you order, and when? Ordering often costs a fixed fee each time. Keeping stock costs money too. The module finds the order size with the **lowest total cost**.

2 WHEN TO USE IT

- Shops and stores that re-order the same item again and again.
- Factories that make items in batches.
- One-time stock decisions with uncertain demand (the **newsvendor**: a daily newspaper that cannot be sold tomorrow).

3 HOW TO ENTER YOUR PROBLEM

1. Open **Inventory** and choose the model in the **Method** switch.
2. Type the numbers shown: **Demand D** (per period), **Set-up cost K** (per order), **Holding cost h** (per unit per period), **Unit cost c**, and optionally the **Lead time L** (same time unit as D).
3. For **EPQ**, add the **Production rate k** (it must be larger than D). For **Shortages**, add the **Shortage cost p**.
4. For **Discounts**, fill the table **From qty / Unit price** (add rows with **+ price break**), choose **All units** or **Incremental**, and give the holding cost as a rate **I** of the price.
5. For the **Newsvendor**, type the selling price, cost and salvage value, and choose the demand: **Normal** (mean and standard deviation), **Uniform** or **Discrete** (a table).

MODEL	IDEA
EOQ	Economic order quantity. Items arrive all at once.
EPQ	Economic production quantity. Items are made at rate k while they are used.
Shortages	You may run out and fill the order later at a penalty p.
Discounts	The price falls when you buy more. The best tier that is feasible wins.
Newsvendor	One period with random demand and leftover stock sold at a salvage price.

4 WHAT YOU GET

- **Policy:** the order quantity, reorder point, cycle length, orders per period, and the cost split into ordering, holding, purchase and total. A short paragraph in plain words, and the derivation with formulas.
- **Cost curve:** total cost against order size, with the best point marked.

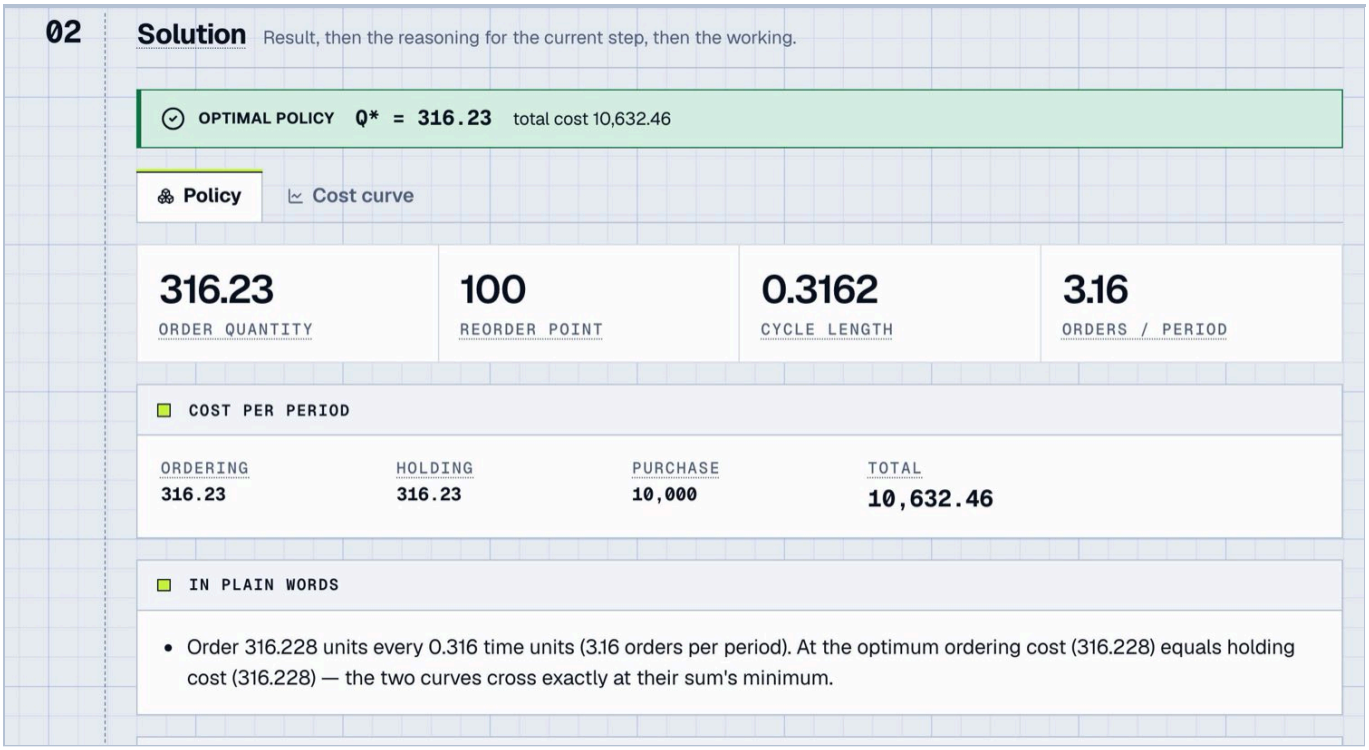


FIG 16.1 · THE POLICY FOR `INV-EOQ`.

5 HOW TO READ THE RESULT

- **Q*** is the best order quantity.
- **Reorder point** is the stock level at which you place the next order, so the new stock arrives as you run out.
- At the best EOQ, **ordering cost equals holding cost**. The curves cross where their sum is lowest.
- For the newsvendor, the **critical ratio** (underage ÷ (underage + overage)) sets the stock level.

6 WORKED MINI EXAMPLE

Library problem `inv-eoq` (Taha 13.3-1): D = 1000 per period, K = 100, h = 2, c = 10, L = 0.1.

Formula	`Q* = √(2KD / h) = √(2·100·1000 / 2)`
Result	`OPTIMAL POLICY` **Q* = 316.23** units, cycle length 0.3162, 3.16 orders per period, **reorder point 100**.
Costs per period	Ordering 316.23 + holding 316.23 + purchase 10,000 = **10,632.46**.

Library problem `inv-news`: price 10, cost 6, salvage 2, demand Normal(100, 20). The critical ratio is (10 − 6) / (10 − 2) = 0.5, so the best stock level is **100**.

7 TIPS AND COMMON MISTAKES

WATCH OUT

- D, K and h must refer to the **same time period** (for example all per year).
- In the EPQ model, the production rate k must be **larger** than the demand D, or the app says *Check inputs*.
- In discount models, enter the holding cost as a **rate** of the price (for example 0.2), not an amount.

GT

17 / ZERO-SUM GAMES

Zero-sum games

Saddle points · dominance · graphical method · exact LP strategies

SHEET 17 OF 25

UNCERTAINTY

1 WHAT IT SOLVES

Two players choose actions at the same time. A table shows what the **row player** wins for each pair of choices. What one player wins, the other loses: this is a **zero-sum** game. The module finds the **best strategy** for each player and the **value of the game**.

2 WHEN TO USE IT

- Competition between two companies, teams or armies.
- Any "I choose, you choose, what do we get" table.

3 HOW TO ENTER YOUR PROBLEM

1. Open **Zero-sum games**. The grid is the **payoff matrix to the row player**.
2. Type the payoffs. A positive number means the row player wins; a negative number means the row player loses.
3. Use **+ / –** next to **Rows** and **Columns** to set the size. Rename the strategies (A1, A2… and B1, B2…).
4. There is no method to choose: the app uses what fits (saddle point, dominance, graphical for 2 × n or m × 2, or linear programming). Use the step player.

4 WHAT YOU GET

A **Solution** tab with the steps (the saddle point check, strikes of dominated rows or columns, the graph or the LP), the **optimal strategies** for both players, the value, and a heatmap of the payoffs. A *Verified* badge shows that the answer was checked. Tutor mode and Diff mode are available.

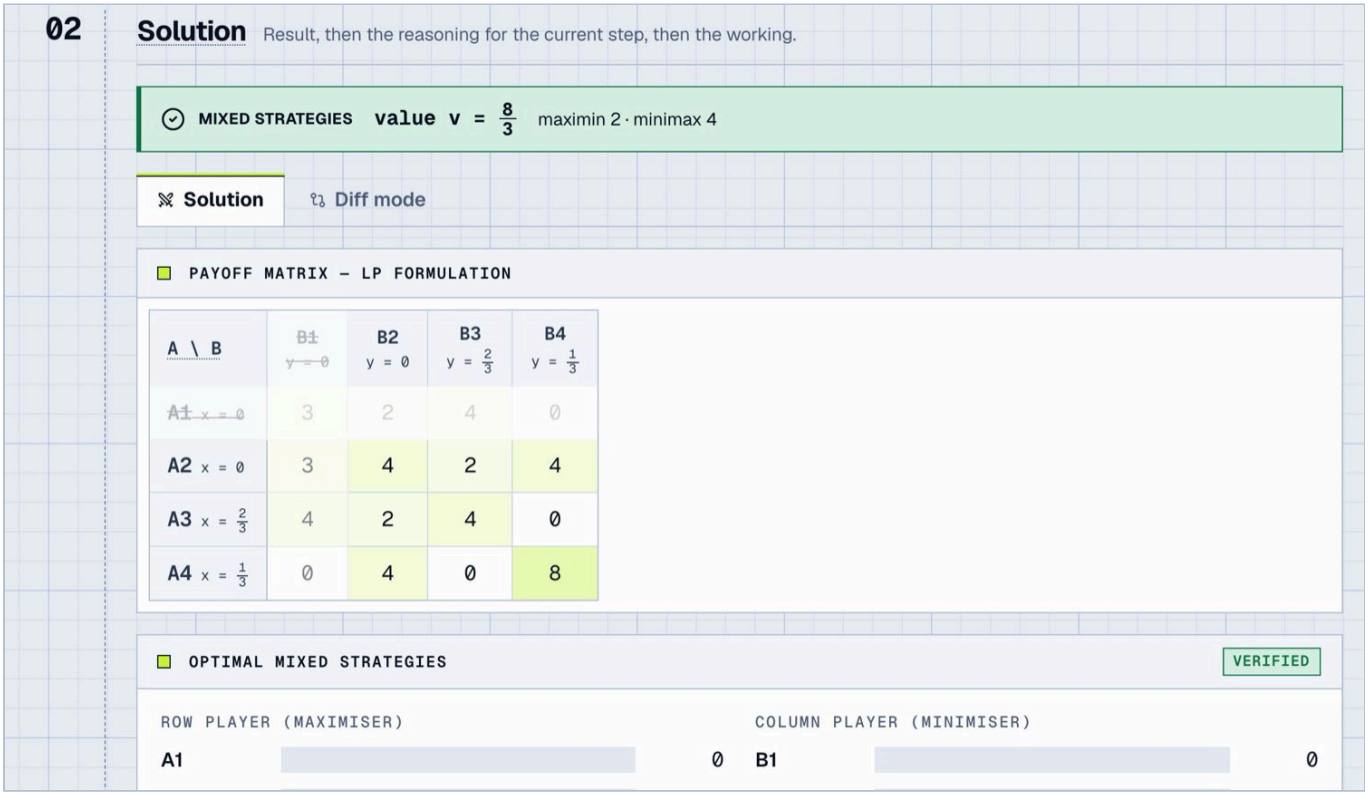


FIG 17.1 · THE LAST STEP OF `G-DOM`: MIXED STRATEGIES FOR BOTH PLAYERS AND THE VALUE 8/3.

5 HOW TO READ THE RESULT

- **Maximin** is the best "worst case" for the row player (the largest of the row minimums). **Minimax** is the best "worst case" for the column player (the smallest of the column maximums).
- If they are **equal**, there is a **saddle point**: each player picks one action for sure (a *pure* strategy).
- If not, each player **mixes**: the numbers show the share of the time to play each action.
- The **value** is what the row player wins on average. A value of 0 means the game is fair.

6 WORKED MINI EXAMPLE

`g-saddle`	Payoffs `3 5 4 / 2 1 0 / 1 6 3`. Row minimums are 3, 0, 1, so maximin = 3. Column maximums are 3, 6, 4, so minimax = 3. Saddle point at (A1, B1), **value 3**.
`g-dom`	A 4 × 4 game. Maximin 2 ≠ minimax 4, so there is no saddle point. After dominance and the LP method: row player plays A3 with ⅔ and A4 with ⅓; column player plays B3 with ⅔ and B4 with ⅓. **Value = 8/3** (about 2.667). It favours the row player.
`g-pennies`	Matching pennies: both mix ½–½, **value 0**.

7 TIPS AND COMMON MISTAKES

WATCH OUT

- Enter payoffs **to the row player**. If your table shows what the *column* player wins, change every sign.
- The app says *No saddle point* in the diagnostics. That is not an error. It means a mixed strategy is needed.

MC

18 / MARKOV CHAINS

Markov chains

n-step probabilities · exact steady state · classes and periodicity · absorption · first passage

SHEET 18 OF 25

UNCERTAINTY

1 WHAT IT SOLVES

A system is always in one of a few **states**. At each step it moves to another state (or stays) with fixed probabilities. The module tells you where it will be after n steps, and what happens in the long run.

2 WHEN TO USE IT

- Weather (sunny or rainy), customers switching brands, machine states (working, broken).
- Random walks such as the **gambler's ruin**, where some states are **absorbing** (you stay there forever).

3 HOW TO ENTER YOUR PROBLEM

1. Open **Markov chains**. In the **Transition probabilities** grid, row *i* and column *j* is the chance to go **from** state *i* **to** state *j*.
2. Each row must add up to 1. The **Σ** column shows each row's total as you type.
3. Change the number of states with the **Size** buttons. Rename the states.
4. Type **Steps n** (0 to 500) and the **Initial distribution** (where the system starts, as probabilities that add up to 1).

4 WHAT YOU GET

- **Analysis:** classification of the states, the steady-state distribution, the matrix P^n, the distribution after n steps, mean first-passage times and, for absorbing chains, the absorption results.
- **Diagram:** the chain drawn as a state diagram, with the probability on each arrow.
- **Convergence:** a chart of the distribution over time, with an **Animate** button. The lines flatten onto the steady state.

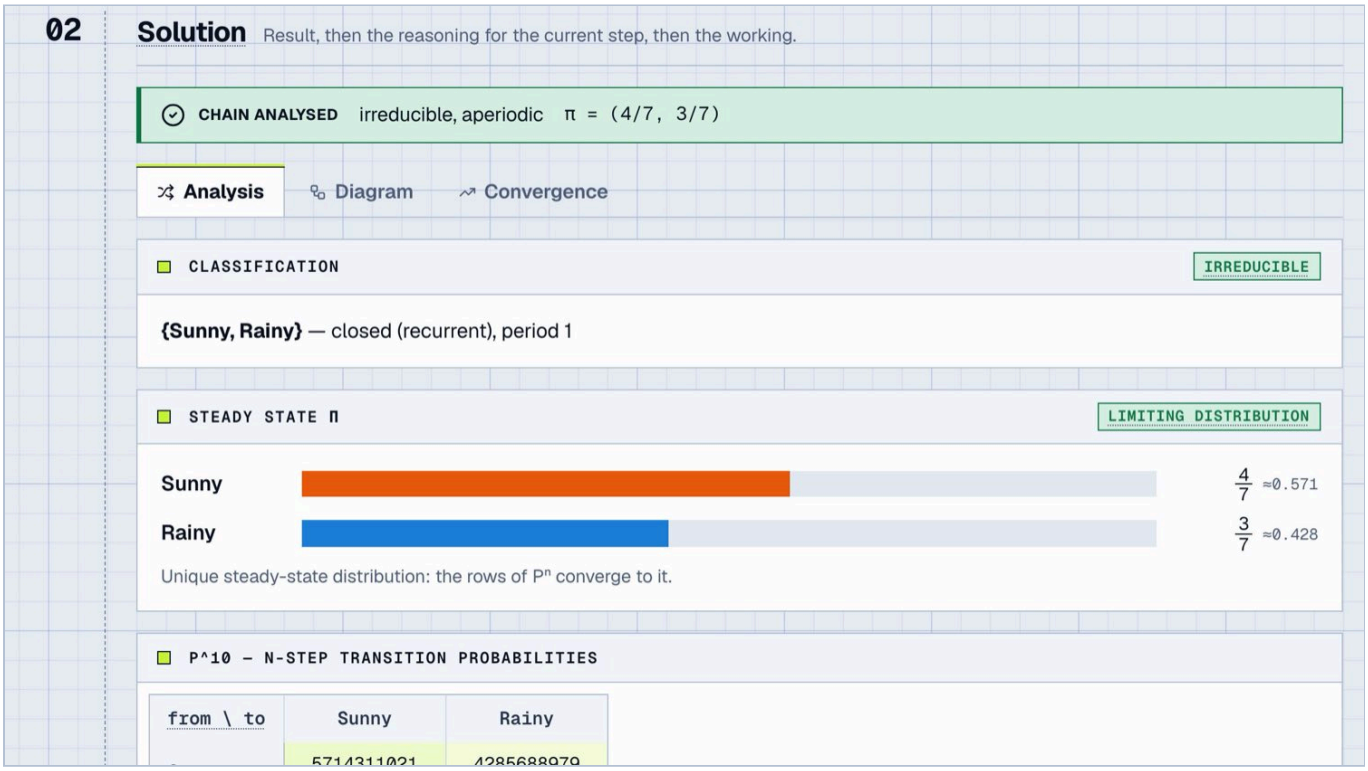


FIG 18.1 · ANALYSIS TAB FOR THE WEATHER CHAIN `MK-WEATHER`.

5 HOW TO READ THE RESULT

- **Irreducible**: every state can be reached from every other. **Aperiodic**: the chain does not run in a fixed cycle.
- **Steady state π** is the share of time spent in each state in the long run. It solves `πP = π` with the shares adding up to 1.
- **Mean first-passage time** is the average number of steps to reach a state for the first time. On the diagonal it is the mean return time, `1/πⱼ`.

6 WORKED MINI EXAMPLE

Library problem `mk-weather`: P = `[[0.7, 0.3], [0.4, 0.6]]`, start Sunny, n = 10. The chain is **irreducible and aperiodic**. The steady state is **π = (4/7, 3/7)**, about 0.571 sunny and 0.429 rainy. After 10 steps from Sunny, the distribution is already 0.5714 and 0.4286. The mean return time to Sunny is 7/4 steps.

Try `mk-abs` (gambler's ruin, with absorbing states Broke and Win) and `mk-period` (a flip-flop of period 2: P^n never settles, yet π = (½, ½) exists).

7 TIPS AND COMMON MISTAKES

WATCH OUT

- Check the **Σ** column: every row must add up to 1.
- A periodic chain has no limiting distribution for P^n, but it still has a steady state.

TIP

Type fractions such as `1/3` to keep the answers exact.

SIM

19 / SIMULATION

Simulation

Monte Carlo · Queue sim · Demand table · Inventory simulation · RNG lab

SHEET 19 OF 25

UNCERTAINTY

1 WHAT IT SOLVES

When a system is too complex for a formula, you can **imitate it on the computer** many times with random numbers, and watch what happens on average. This module has five tools, chosen in the **Method** switch.

TOOL	WHAT IT DOES	RESULT TABS
Monte Carlo	Estimates an average (or a probability) by trying a formula on many random inputs.	Mean, spread, confidence interval, histogram, convergence plot
Queue sim	Runs a waiting line with random arrivals and service, one customer at a time.	Results, Event trace
Demand table	Simulates daily demand from a probability table, using random-number intervals (the classroom method).	Walkthrough, Simulation table, Summary
Inventory	Simulates a periodic-review stock policy day by day and searches for the best order-up-to level S.	Walkthrough, Day table, Policy search, Totals
RNG lab	Shows how random numbers are made (mid-square, congruential, Mersenne Twister) and tests them.	Working table, cycle and period, tests, plot

2 WHEN TO USE IT

- No simple formula exists, or you want to check one (compare the Queue sim with the Queuing module).
- Your course asks for **hand simulation with random-number tables** (Unit 7, Experiment 10).

3 HOW TO ENTER YOUR PROBLEM

1. Open **Simulation** and choose a tool.
2. **Monte Carlo and Queue sim** start with the **Random numbers** card: a **Seed** (the same seed always gives the same results), the **Generator** (Mersenne Twister or LCG 16807), **New random seed** and **Re-run**.
3. **Monte Carlo:** add random variables with **+ random variable**. Choose each one's distribution (uniform, exponential, normal, triangular, Poisson, discrete or constant) and its parameters. Type the **Expression** that combines them, the number of **Trials** and, if you like, a **Threshold** to estimate the chance that the result is at least that value.
4. **Queue sim:** choose the **Interarrival** and **Service** distributions, the number of **Servers**, **Customers**, and a **Warm-up** (customers ignored at the start).
5. **Demand table:** fill **Demand / Probability** rows (they must total 1). Choose **Given list** to type your own random numbers, or **Generate**. Set the **Digits per random number** and the **Interval convention** (00–99 or 01–100).
6. **Inventory:** type the policy (**Order-up-to level S**, **Review every R days**, **Initial stock**, ordering, holding and shortage costs, **Lost sales** or **Backorders**), the demand table, the lead time (**Fixed** or **Random**) and the random numbers. **Policy search** tries a range of S.
7. **RNG lab:** choose a **Method**, type its numbers (for the congruential method: **a, c, m** and the seed X_0, or a preset such as *Short cycle*) and a **Sample size**.

4 WHAT YOU GET



FIG 19.1 · MONTE CARLO (`SIM-PI`): THE ESTIMATE, ITS CONFIDENCE INTERVAL, THE HISTOGRAM AND THE CONVERGENCE PLOT.

5 HOW TO READ THE RESULT

- **Mean** is the estimate. The **95 % confidence interval** is the range that very likely holds the true value. It **shrinks like 1/√n**: four times the trials halve the width.
- The **convergence plot** shows the estimate settling as trials grow. In the Queue sim, a line shows the formula's value so you can compare.
- The **Demand table** and **Inventory** tools show every day in a table, so you can check your hand work row by row.
- In the **RNG lab**, the **period** is how many numbers come before the sequence repeats, and the **chi-square** test checks that the numbers spread evenly.

6 WORKED MINI EXAMPLES

`sim-pi`	Estimate π: random points (x, y) in a square, count those inside the quarter circle. With seed 12345 and 20,000 trials: **mean ≈ 3.138**, 95 % interval [3.11521, 3.16079].
`sim-queue`	An M/M/1 queue with λ = 2 and μ = 3, 5,000 customers. Simulated waiting time **Wq ≈ 0.6229**; the formula gives 0.6667.
`sim-inv-sS`	Ten days, order up to S = 8, review every 5 days. **Total cost 76** (7.6 per day), fill rate 61.9 %. The policy search finds the cheapest S = 11.
`sim-rng-lcg`	`X(n+1) = (17·X(n) + 43) mod 100` from X_0 = 27 gives 2, 77, 52, 27 and then repeats. **Period 4**: a poor generator.
`sim-exp10-demand`	Ten days of cake demand: see Walk-through B.

7 TIPS AND COMMON MISTAKES

WATCH OUT

- A simulation gives an **estimate**, not the exact answer. A 10-day simulation is very noisy; do not expect it to match the formula.
- Click **New random seed** and **Re-run** to see how much results change. The same seed always repeats exactly.
- In the demand table the probabilities must **total 1**. The table shows *Total 1* when they do.
- The number of digits in each random number must match your table (2 digits for 00 to 99).

SEQ

20 / JOB SEQUENCING

Job sequencing

Johnson's rule · three machines · CDS heuristic · exact search · Gantt chart and idle times

SHEET
20 OF 25
FLOWS AND PLANS

1 WHAT IT SOLVES

Several **jobs** must go through the same **machines** in the same order (for example first the saw, then the paint). In what **order** should the jobs enter so that everything is finished as early as possible? The time from the start of the first job to the end of the last job is the **total elapsed time** (also called the makespan).

2 WHEN TO USE IT

- Workshops, printing, assembly and bakery lines.
- Course Unit 7 and Experiment 10 (activities 4 and 5).

3 HOW TO ENTER YOUR PROBLEM

1. Open **Job sequencing**. In the grid, each **row is a job** and each **column is a machine**, in the order the jobs visit them. Type the processing times.
2. Add or remove jobs and machines with **+ / −**. Rename a job or machine by typing in its header cell. **Clear** empties the grid.
3. Choose the method in the title block (see the table).
4. Use the step player to watch the sequence being built, then open the **Gantt** tab.

METHOD	USE IT FOR
Johnson	Two machines. It always gives the best sequence. It also works for **three machines** if a simple condition holds (the smallest time on machine 1 is at least the largest on machine 2, or the same for machine 3).
CDS	Three or more machines. A good and fast **heuristic** (a rule of thumb) from Campbell, Dudek and Smith. Not proven best.
Exact	Tries **every** order and keeps the best. Works for up to 9 jobs.

4 WHAT YOU GET

- **Sequence:** the best order, the total elapsed time and the **idle time** of each machine.
- **Gantt:** a bar chart of when each job runs on each machine.
- **Table:** the in and out times of each job on each machine.
- **Compare:** the methods side by side, so you can see how far a heuristic is from the best.
- Tutor mode asks you to place each job.

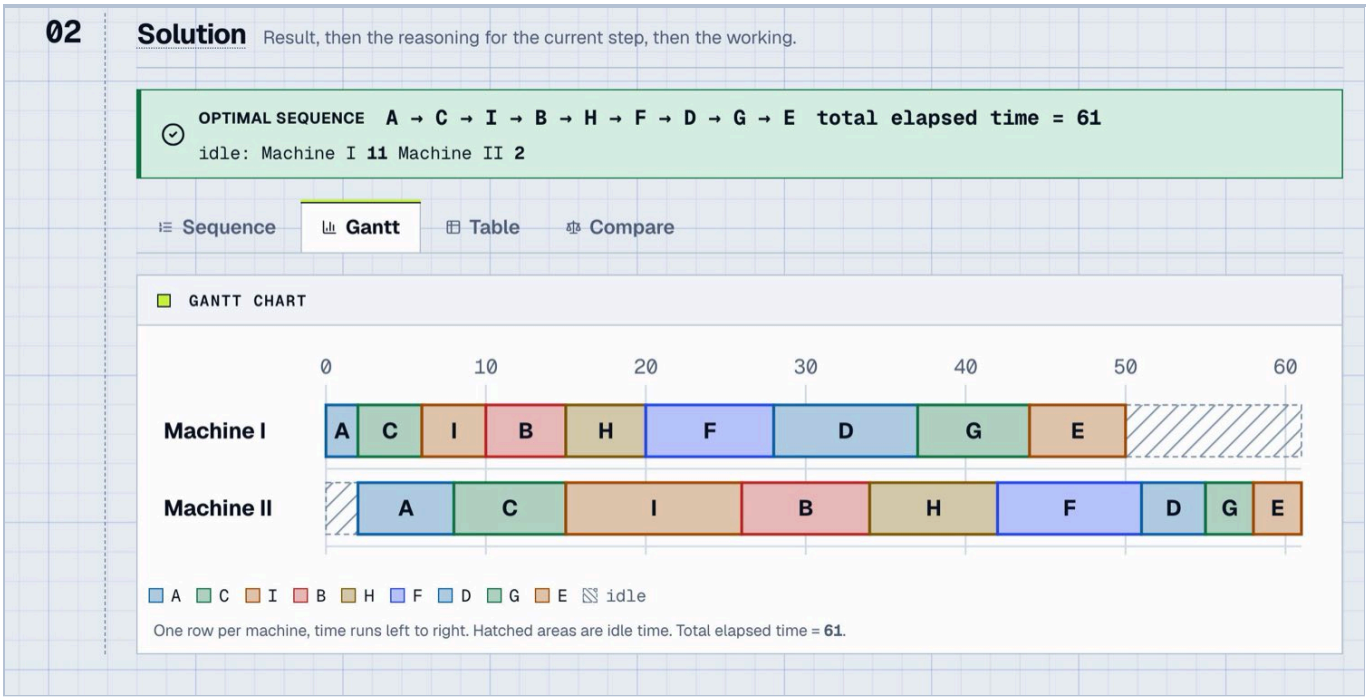


FIG 20.1 · THE GANTT CHART OF `SEQ-EXP10`. GAPS ON A MACHINE ARE IDLE TIME.

5 HOW TO READ THE RESULT

- **Johnson's rule:** find the smallest time in the table. If it is on the **first** machine, put that job as **early** as possible. If it is on the **second** machine, put it as **late** as possible. Remove the job and repeat.
- **Total elapsed time** = the time the last job leaves the last machine.
- **Idle time** of a machine = total elapsed time − the sum of its processing times.
- A banner of *Heuristic sequence — not proven optimal* means the result of CDS or of a three-machine Johnson that may not be best. Compare it with Exact.

6 WORKED MINI EXAMPLE

Library problem `seq-exp10` is the course example (Walk-through C): 9 jobs on 2 machines, best sequence **A → C → I → B → H → F → D → G → E**, total elapsed time **61 hours**, idle **11 hours** on Machine I and **2 hours** on Machine II.

Library problem `seq-4m-cds` (four machines, jobs A to E): CDS builds three candidate sequences with total times 52, 53 and 51. The best is **E → A → C → D → B, total 51**, with idle time 18, 19, 31 and 19 hours on M1 to M4.

7 TIPS AND COMMON MISTAKES

WATCH OUT

- Machines must be in the **order of work**, from left to right. Every job uses the same order.
- With three machines, check the banner. If the Johnson condition fails, the result is only a heuristic. Try **Exact**: see `seq-3m-nocond`, where a better sequence exists.
- **Ties** are normal. Different sequences can give the same total (see `seq-ties`: total 18). Any one of them is correct.

A

A / WORKED WALK-THROUGH

Reddy Mikks, from typing to shadow prices

One linear programme, start to finish. Follow it on your own screen.

SHEET A OF C

LINEAR PROGRAMMING

The problem. A paint company makes exterior paint (x_1) and interior paint (x_2) from two raw materials, M1 and M2. Each ton of exterior paint earns 5 (thousand dollars) and each ton of interior paint earns 4. The company has 24 tons of M1 and 6 tons of M2 each day. Exterior paint needs 6 tons of M1 and 1 ton of M2 per ton; interior paint needs 4 tons of M1 and 2 tons of M2. The market limits the daily excess of interior over exterior paint to 1 ton, and interior paint sells at most 2 tons a day. **How much of each paint should be made to earn the most?**

1 **Open the module.** Click **LP** in the left rail. The screen opens with an example in the Model box.

2 **Type the model.** Click in the text box, select all (`Ctrl/⌘` + `A`) and type:

```
max 5x1 + 4x2
subject to
  M1: 6x1 + 4x2 <= 24
  M2: x1 + 2x2 <= 6
  Market: -x1 + x2 <= 1
  Demand: x2 <= 2
  x1, x2 >= 0
```

The line under the box turns green: *2 variables, 4 constraints — parsed.* (Or pick *Reddy Mikks paint company* from the **Example** list.)

3 **Read the banner.** It says `OPTIMAL` `max z = 21`, *Standard simplex · 2 pivots*. The **Method** is Auto. All rows are ≤ with positive right-hand sides, so the plain simplex was chosen.

4 **Walk through the steps.** Press `Home` to go to step 1, then `→` to move on. There are 4 steps.

STEP	NAME	WHAT HAPPENS
1	Initialisation	Each constraint gets a slack variable (s_1 to s_4). The slacks form the starting basis. This is the origin, x = 0 and z = 0.
2	Simplex iteration	**x_1 enters** (reduced cost −5 is the most negative). Ratio test: 4, 6, —, —. Smallest is 4, so **s_1 leaves**. Pivot element = 6. z goes from 0 to 20.
3	Simplex iteration	**x_2 enters** (reduced cost −2/3). Ratios 6, 3/2, 3, 2. Smallest is 3/2, so **s_2 leaves**. Pivot element = 4/3. z goes from 20 to 21.
4	Termination	No negative number is left in the z row, so the solution is **optimal**: $x_1 = 3$, $x_2 = 3/2$, z = 21.

5 **Read the Solution card** below the last tableau. Variables: `x₁ = 3`, `x₂ = 3/2`, `z = 21`. Constraints: M1 and M2 are marked `TIGHT` (slack 0). Market has slack 5/2 and Demand has slack 1/2.

6 **Find the shadow prices** in the last column of the same card: **M1 = 3/4** and **M2 = 1/2**. Market and Demand are 0.

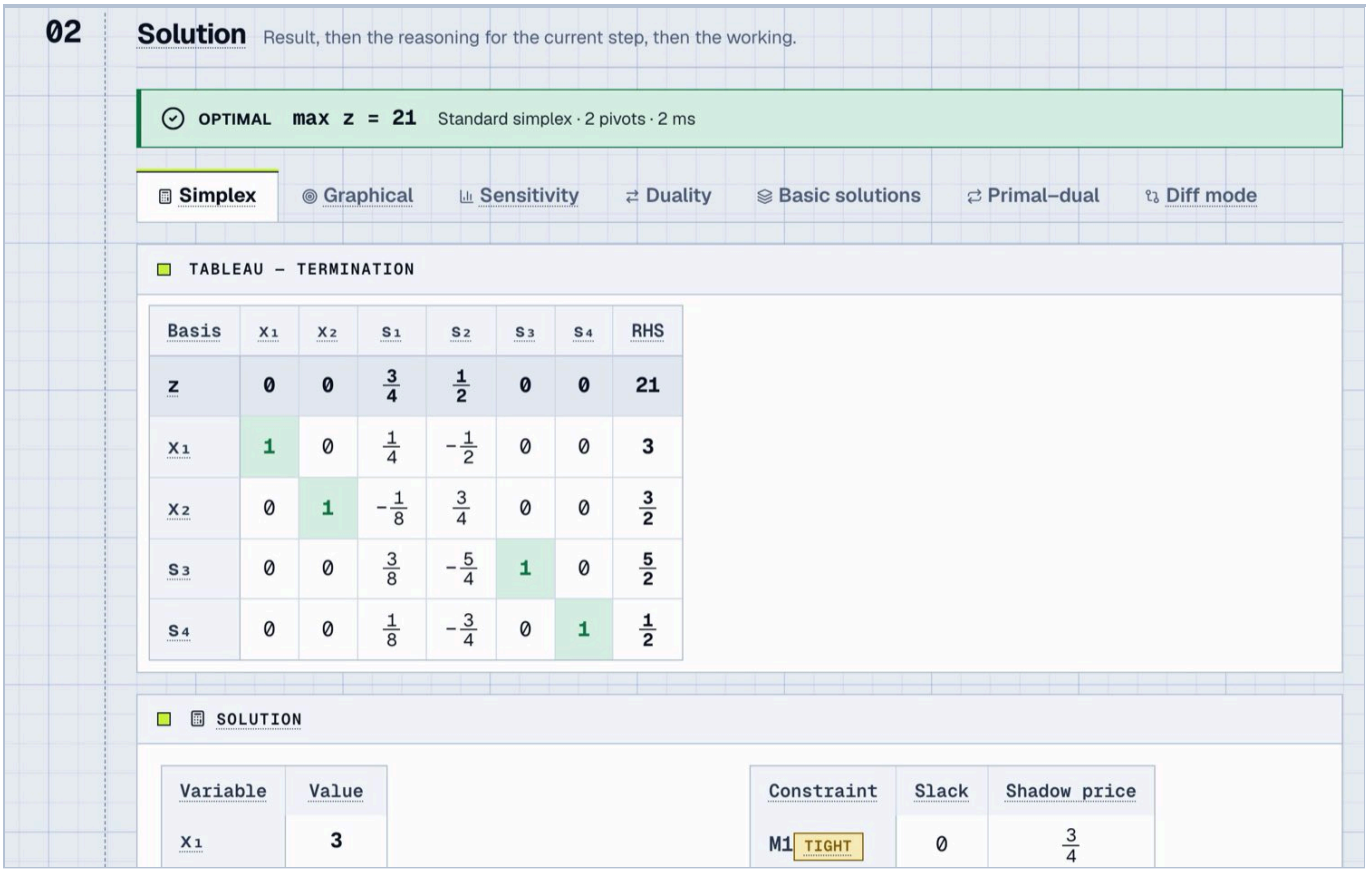


FIG A.1 · STEP 4: THE FINAL TABLEAU, AND BELOW IT THE SOLUTION CARD WITH THE SHADOW PRICES.

WHAT THE SHADOW PRICES MEAN

One more ton of raw material M1 raises the daily profit by **3/4** (750 dollars). One more ton of M2 raises it by **1/2**. More Market or Demand room is worth nothing, because those limits are not tight. If you could buy extra M1 at a price below 0.75 per ton, it would pay.

7 **Check the range.** Open the **Sensitivity** tab. For M1 the green bar runs from 20 to 36. Inside this range the shadow price 3/4 stays true. For M2 it runs from 4 to 20/3. Move the *what-if RHS* slider and watch the sentence change.

8 **Test it.** Change 24 to 26 in the M1 row of the model. The answer becomes `z = 22.5` at x = (3.5, 1.25). That is 21 + 2 × 3/4, exactly as predicted. Change it back to 24.

9 **See it on a graph.** Open the **Graphical** tab. The best point (3, 3/2) is where the lines M1 and M2 cross. These are the two tight constraints.

10 **Keep the work.** Click the download icon and choose **PDF — full solution**, or click **Save** and name the model (chapter 7).

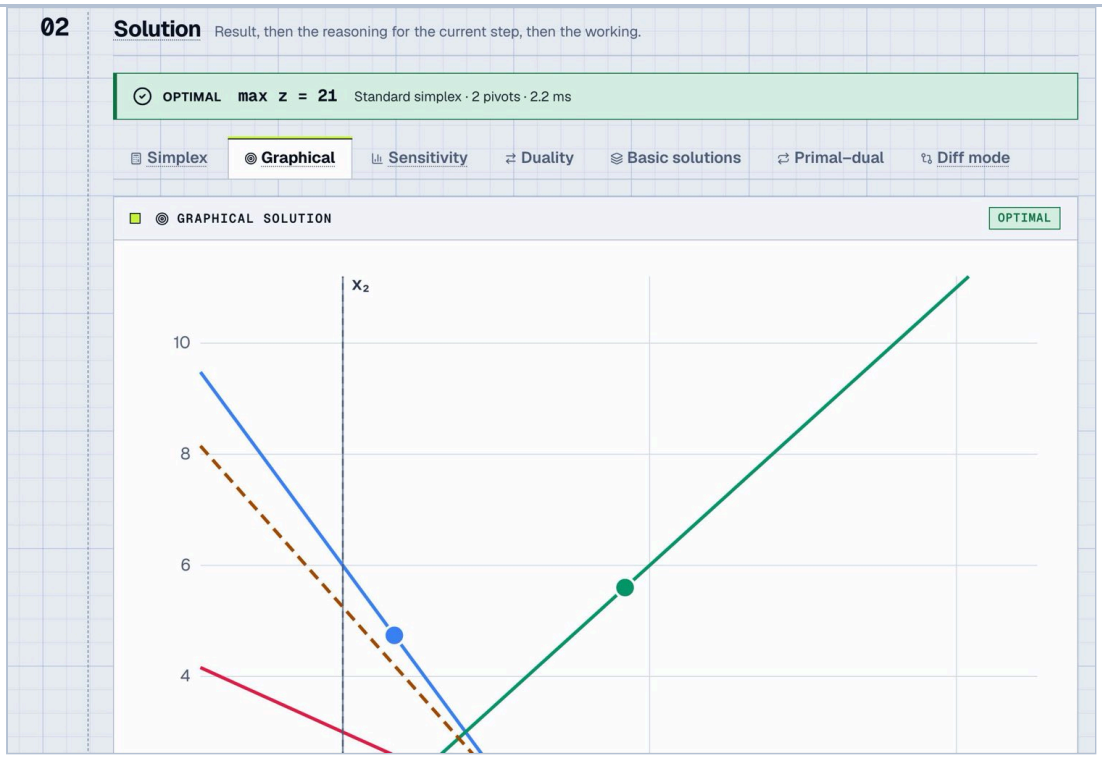


FIG A.2 · STEP 9: THE GRAPHICAL TAB. THE BEST POINT IS WHERE THE TWO TIGHT CONSTRAINTS CROSS.

ANSWER IN ONE LINE

Make **3 tons** of exterior paint and **1.5 tons** of interior paint each day for a daily profit of **21** (thousand dollars). Raw materials M1 and M2 are the bottleneck, worth 3/4 and 1/2 per extra ton.

B

B / WORKED WALK-THROUGH

Experiment 10, activity 1: demand simulation

Ten days of cake demand in a stationery shop, with the given random numbers.

SHEET

B OF C

SIMULATION

The problem. A stationery shop sells cakes. Daily demand is not fixed. From past records, the demand and its chance are:

DEMAND	0	10	20	30	40	50
Probability	0.01	0.20	0.15	0.50	0.12	0.02

Simulate ten days with these random numbers (two digits each): `40, 19, 87, 83, 73, 84, 29, 09, 02, 20`.

1. Open **Simulation** (**SIM** in the rail). In the **Method** switch choose **Demand table**.
2. Fill the **Demand / Probability** table (or choose *Stationery shop: ten days of cake demand* in the **Example** list). The note shows *Total 1*.
3. Under **Random numbers and intervals** choose **Given list** and type the ten numbers. Set **Digits per random number** to 2 and the **Interval convention** to **00–99**. The app reads 10 numbers, so it simulates 10 days.
4. Open the **Walkthrough** tab and press → to go through the 13 steps.

1 CUMULATIVE PROBABILITY AND INTERVALS (STEPS 1 AND 2)

Add the probabilities down the table to get the **cumulative probability**. Multiply by 100 and round (a half rounds up) to get the **random-number interval** for each demand. A demand with probability p gets about $p \times 100$ of the 100 random numbers 00 to 99.

DEMAND	PROBABILITY	CUMULATIVE	RANDOM-NUMBER INTERVAL
0	0.01	0.01	**00**
10	0.20	0.21	**01 – 20**
20	0.15	0.36	**21 – 35**
30	0.50	0.86	**36 – 85**
40	0.12	0.98	**86 – 97**
50	0.02	1.00	**98 – 99**

2 THE TEN DAYS (STEPS 3 TO 12)

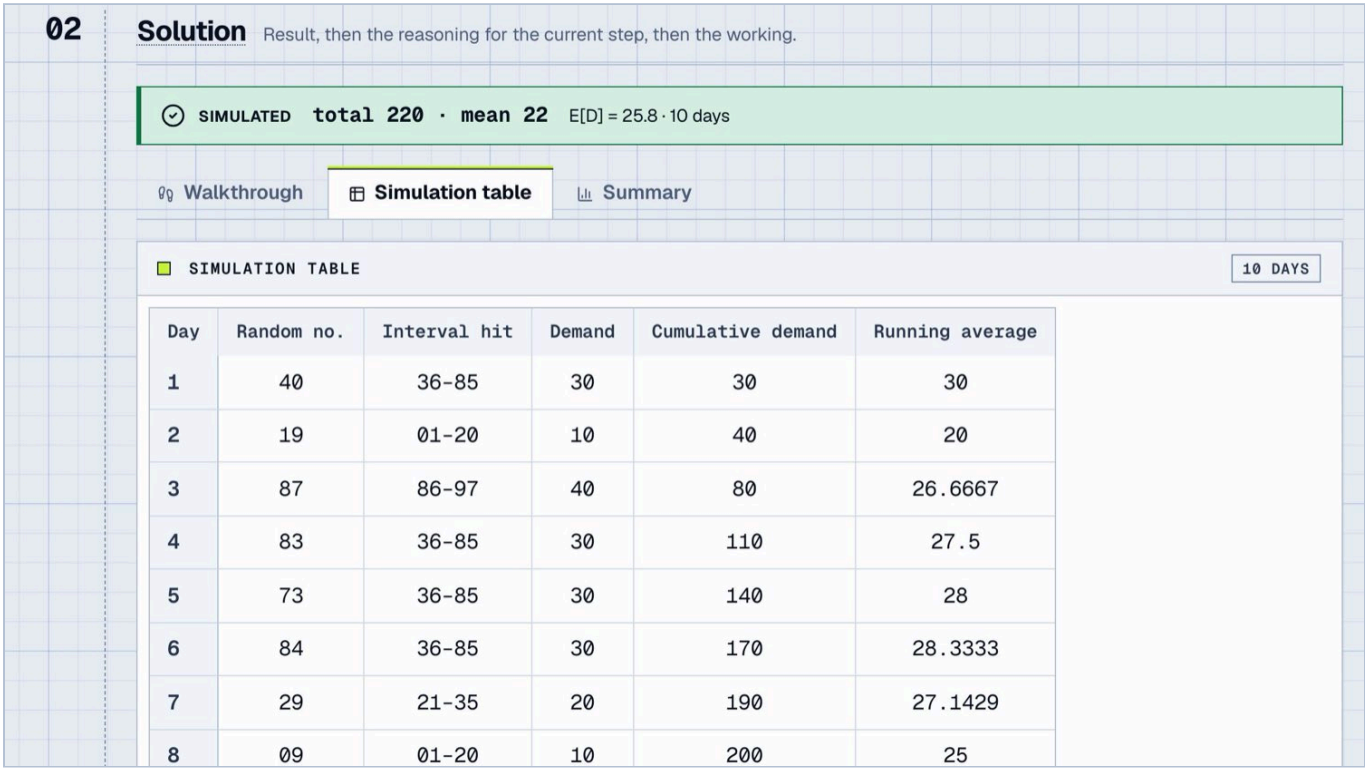
For each day, find the interval that holds the random number. The demand of that interval is the demand for the day. For day 1, the number 40 lies in 36–85, so the demand is 30.

DAY	RANDOM NO.	INTERVAL HIT	DEMAND	CUMULATIVE DEMAND	RUNNING AVERAGE
1	40	36–85	30	30	30
2	19	01–20	10	40	20
3	87	86–97	40	80	26.6667
4	83	36–85	30	110	27.5
5	73	36–85	30	140	28
6	84	36–85	30	170	28.3333
7	29	21–35	20	190	27.1429
8	09	01–20	10	200	25
9	02	01–20	10	210	23.3333

DAY	RANDOM NO.	INTERVAL HIT	DEMAND	CUMULATIVE DEMAND	RUNNING AVERAGE
10	20	01–20	10	**220**	**22**

3 THE SUMMARY (STEP 13)

- The simulated demand is **220 cakes** in 10 days, an average of **22 per day**.
- The theoretical (expected) demand is `E[D] = Σ x·p = 25.8` per day.
- The gap is **−3.8**. Ten days is a very small sample. The gap shrinks roughly like 1/√n as you simulate more days.



Day	Random no.	Interval hit	Demand	Cumulative demand	Running average
1	40	36-85	30	30	30
2	19	01-20	10	40	20
3	87	86-97	40	80	26.6667
4	83	36-85	30	110	27.5
5	73	36-85	30	140	28
6	84	36-85	30	170	28.3333
7	29	21-35	20	190	27.1429
8	09	01-20	10	200	25

FIG B.1 · THE **SIMULATION TABLE** TAB: THE SAME TABLE AS ABOVE, WITH THE TOTALS UNDERNEATH.

TRY THIS

Choose **Generate** instead of **Given list** and increase the number of days. The simulated mean moves towards 25.8. Compare it with your hand result for 10 days.

WATCH OUT

If your textbook uses the convention **01–100** (so that 00 stands for 100), change the **Interval convention** to **01–100**. The intervals shift by one number and a different day may fall in another interval.

C

C / WORKED WALK-THROUGH

Experiment 10, activity 4: Johnson's rule

Nine jobs on two machines: the best order, the total time and the idle time.

SHEET C OF C

JOB SEQUENCING

The problem. Nine jobs A to I each need Machine I first and then Machine II. The times (hours) are:

JOB	A	B	C	D	E	F	G	H	I
Machine I	2	5	4	9	6	8	7	5	4
Machine II	6	8	7	4	3	9	3	8	11

In what order should the jobs run to finish as early as possible? What are the total elapsed time and the idle times?

1. Open **Job sequencing** (**SEQ**). Choose the method **Johnson**.
2. Set the grid to **9 rows** and **2 columns** and type the times. Rename the columns *Machine I* and *Machine II*. (Or pick *Nine jobs, two machines (Experiment 10)* in the **Example** list.)
3. Use the step player. There are 12 steps. Steps 2 to 10 place the jobs one by one.

1 BUILDING THE SEQUENCE WITH JOHNSON'S RULE

At each step, find the smallest time among the jobs not yet placed. If it is on **Machine I**, put that job in the **first free slot** from the front. If it is on **Machine II**, put it in the **last free slot** from the back. Ties can be broken in any way.

STEP	SMALLEST TIME LEFT	ACTION
2	2: job A on Machine I	A goes to the front, slot 1
3	3: job E on Machine II	E goes to the back, slot 9 (G also has 3: a tie)
4	3: job G on Machine II	G goes to slot 8
5	4: job C on Machine I	C goes to slot 2
6	4: job I on Machine I	I goes to slot 3
7	4: job D on Machine II	D goes to slot 7
8	5: job B on Machine I	B goes to slot 4
9	5: job H on Machine I	H goes to slot 5
10	8: job F on Machine I	F goes to slot 6

THE SEQUENCE

A → C → I → B → H → F → D → G → E

2 THE TIMING (STEP 11)

A job starts on a machine when both the previous job has left that machine and this job has left the previous machine. So `in = max(out of previous job on this machine, out of this job on the previous machine)` and `out = in + time`.

JOB	A	C	I	B	H	F	D	G	E
M I in–out	0–2	2–6	6–10	10–15	15–20	20–28	28–37	37–44	44–50
M II in–out	2–8	8–15	15–26	26–34	34–42	42–51	51–55	55–58	58–61

3 THE RESULT (STEP 12)

Total elapsed time = 61 hours. The last job (E) leaves Machine II at hour 61.

- **Idle time of a machine** = total elapsed time − the sum of its processing times.
- Machine I works 2+5+4+9+6+8+7+5+4 = 50 hours, so it is idle **61 − 50 = 11 hours** (waiting at the end, after job E).
- Machine II works 6+8+7+4+3+9+3+8+11 = 59 hours, so it is idle **61 − 59 = 2 hours** (waiting at the start, while job A is on Machine I).

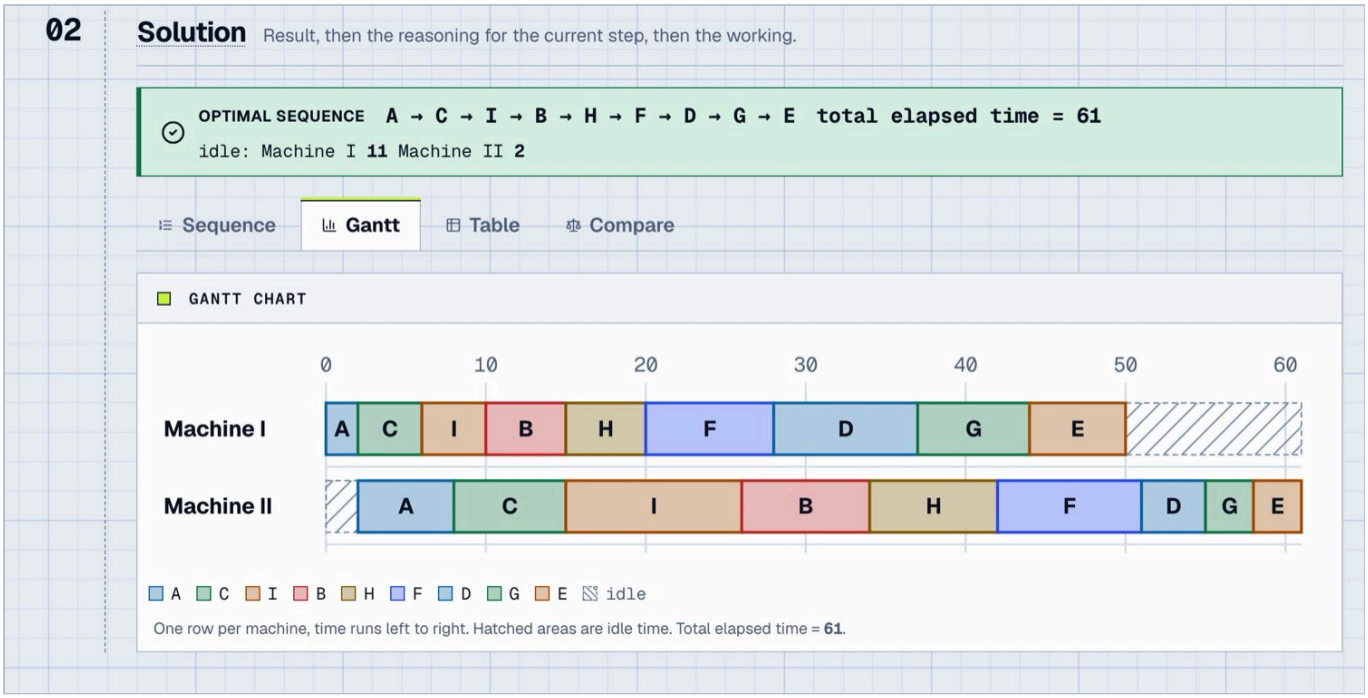


FIG C.1 · THE **GANTT** TAB. MACHINE II NEVER WAITS AFTER HOUR 2. MACHINE I IS FREE FOR THE LAST 11 HOURS.

GOOD TO KNOW

Johnson's rule gives a best (minimum-time) sequence for two machines, but it is not the only one. Many other orders also reach 61 hours (the app counts 12,344 of the 362,880 possible orders). Use the **Compare** tab to check.

21

21 / READING RESULTS

Reading results

What Infeasible, Unbounded, Degenerate, Alternate optima and Iteration limit mean, and what to do.

SHEET 21 OF 25

REFERENCE

Not every problem has one neat answer. OR/Studio names each special case and explains it. These are the ones you will meet most.

MESSAGE	WHAT IT MEANS	WHAT TO DO
INFEASIBLE	No answer can satisfy every constraint at once. The app shows which constraints conflict, and a combination of them that gives a false statement (such as 0 ≥ 3). Example `lp-infeasible`: `x1 + x2 <= 2` and `x1 + x2 >= 5`.	Look for a wrong sign, a wrong number, or a limit that is too tight. Relax one of the conflicting constraints.
UNBOUNDED	The objective can grow forever, because nothing stops it. The app names a direction in which you can keep improving. Example `lp-unbounded`: `max 2x1 + x2` with only `x1 - x2 <= 10`.	A limit is missing. Add the constraint that a real resource would give (time, money, material), or check a wrong ≤ or ≥.
DEGENERATE	A basic variable is exactly 0, so several bases describe the same corner. Pivots can then leave the objective unchanged. Example `lp-beale`. In Transportation it appears as a used cell with 0 units (shown as ε).	Nothing is wrong. The answer is valid. The app handles ties with a safe rule and watches for cycling.
ALTERNATE OPTIMA	More than one answer gives the same best value. A non-basic variable has a reduced cost of exactly 0. Every point on the edge between two optimal corners is optimal too. Example `lp-alt`: z = 10 from (0, 5/2) to (3, 1).	Choose the answer that suits you best for other reasons (cost, ease). Open the second vertex from the Solution card.
ITERATION LIMIT	The solver stopped after a fixed number of steps (1000 for simplex) to protect your browser. The last tableau is shown, but the app says **optimality is not claimed**.	Do not trust the value. Check the model for typing mistakes or huge numbers, simplify it, or try another method.
REDUNDANT CONSTRAINT	A constraint is already guaranteed by the others. It never limits the answer.	You may delete it. It is not an error.
INTEGER INFEASIBLE	The relaxed LP has an answer but no whole-number point does. Example `int-infeasible`: `2x1 = 3`.	Relax the whole-number rule, or change a constraint.
NO SADDLE POINT	In a game, maximin and minimax differ. The players must mix their actions.	Read the mixed strategies in the result.
NOT STABLE (P ≥ 1)	In a queue without a capacity limit, customers arrive faster than the servers can serve them.	Add servers or lower the arrival rate.
CYCLIC DEPENDENCY	In a project, activity A waits for B and B waits for A (directly or through others). The app names the loop.	Remove one of the links in the loop.
CHECK INPUTS	A number does not make sense for the model, for example an EPQ production rate that is not larger than the demand.	Read the message under the banner and fix the input it names.

NOTE

The **Diagnostics** card in the Model area lists notes about the model: for example *degenerate pivot*, *tie in the ratio test*, *new incumbent*. Many of them are for information only.

22

22 / GLOSSARY OF SHORT FORMS

Glossary of short forms

The same meanings you see when you hover over a word in the app.

SHEET
22 OF 25
REFERENCE

This list is built from the app's own dictionary (273 entries), so it matches the hover help. Where the same letters mean different things in two modules, the entry names the module.

NAMES OF VARIABLES IN A TABLEAU

x_1, x_2, …	Decision variables: quantities you choose (for example units of a product). The solver finds their best values.
s_1, s_2, …	Slack variables: the unused amount of a "≤" constraint. It turns the inequality into an equation. Slack 0 means the constraint is fully used.
e_1, e_2, …	Surplus variables: the amount by which a "≥" constraint is exceeded.
a_1, a_2, …	Artificial variables: temporary variables added to get a starting solution for "≥" or "=" constraints. They must end at 0 for the solution to be feasible.
S1, S2 / D1, D2	In Transportation: sources (supply points, rows) and destinations (demand points, columns).
M1, M2, …	In Job sequencing: the machines every job must pass through, in this order.

LINEAR PROGRAMMING / SIMPLEX

RHS	The constant on the right of each constraint (the available amount of a resource). In a tableau it is the current value of each basic variable.	**Extreme point**	A corner of the feasible region that cannot be written as a mix of two other feasible points. Every extreme point is a basic feasible solution.
Basis	The variables that are "in the solution" right now (basic variables). Their columns form an identity matrix; all other variables are zero.	**Primal**	The original linear program. Every primal has a mirror problem called the dual.
z / Z	The value of the objective function (profit to maximise or cost to minimise). In the tableau the z row shows reduced costs; its last entry is the current value.	**Dual**	The mirror linear program built from the primal: constraints become variables and variables become constraints. Both have the same optimal value (strong duality). As a method name: dual simplex.
Ratio / ratio	RHS ÷ the entering column's entry, only for positive entries. The smallest ratio decides which variable leaves, so no variable becomes negative.	**Complementary slackness**	At the optimum, a constraint has slack only if its dual variable is 0, and a variable is positive only if its dual constraint is tight.
Slack	Unused amount of a "≤" resource: how much of the limit is left over. Slack 0 means the constraint is binding (fully used).	**Simplex**	Moves from one corner of the feasible region to a better neighbouring corner until no neighbour improves the objective.
Binding	A constraint that holds with equality at the optimum: the resource is fully used, so it limits the objective and has a positive shadow price.	**Two-Phase**	Phase 1 finds a starting feasible solution by minimising artificial variables; phase 2 then optimises the real objective.
tight	Same as binding: the constraint is satisfied exactly (no slack).	**Big-M**	Handles ≥ and = constraints by adding artificial variables with a huge penalty M in the objective. Here M is kept as a symbol, never a number.
Shadow price	How much the optimal objective improves if the right-hand side of this constraint increases by one unit. Zero for constraints that are not binding.	**Graphical**	Solves two-variable problems by drawing the constraints, shading the feasible region and finding the best corner.
Range	The interval over which the stated value (a cost coefficient or a right-hand side) can change while the current optimal basis stays optimal.	**Sensitivity**	Shows how far costs or right-hand sides can change before the optimal solution (or its basis) changes.
Value	The current numerical value of the variable in this solution.	**Diff mode**	Enter your own working; the app finds the first place it differs from the solver and infers the likely mistake.
Variable	A decision variable (something you choose) or a generated slack, surplus or artificial variable.	**Tutor**	The solver pauses and asks you to choose the next move. Wrong answers are explained.
Constraint	A limit the solution must respect, such as a resource availability or a demand requirement.	**Auto**	The solver runs by itself and you step through every move with the player.
Status	Whether the constraint or solution is binding, slack, optimal, feasible or infeasible.	**Hyperplane**	The set of points where a linear equation holds exactly (a line in 2-D, a plane in 3-D). Each constraint boundary is a hyperplane.
Δ cost / unit	Extra cost of shortening an activity by one time unit (crash cost per unit time).	**Convex set**	A region where the straight line between any two of its points stays inside it. The feasible region of a linear program is convex.

Vertex	A corner of the feasible region. The optimum of a linear program is always at a vertex (or along an edge between two).	**Degenerate / degenerate**	A basic solution in which a basic variable equals 0, so more than the minimum number of constraints meet at the corner. It can cause ties and, rarely, cycling.
BFS	A solution where n − m variables are zero and the rest (the basis) satisfy the constraints and are ≥ 0. Geometrically it is a corner (extreme point) of the feasible region.	**M**	A symbolic penalty, larger than any real number in the problem, charged to artificial variables so the simplex drives them out.

INTEGER PROGRAMMING

Node	A point of a network, or — in branch and bound — one sub-problem: the original problem plus the branching limits added so far.	**Incumbent**	The best integer-feasible solution found so far. Any branch whose bound is not better can be pruned.
Branch & bound	Solve the relaxed problem, split on a fractional variable into two sub-problems (≤ floor, ≥ ceiling), and discard branches that cannot beat the best integer solution.	**LP bound**	The optimum of the relaxed problem (integrality ignored). No integer solution can be better than it.
Gomory cuts	Extra constraints derived from a fractional row of the tableau that cut off the fractional optimum but keep every integer solution.		

TRANSPORTATION / ASSIGNMENT

u TRANSPORT	MODI value for each source row. For every occupied cell $u_i + v_j = c_{ij}$.	**NWC**	Starting solution: fill the top-left cell as much as possible, then move right or down. Fast but ignores costs.
v TRANSPORT	MODI value for each destination column. For every occupied cell $u_i + v_j = c_{ij}$.	**Least-cost**	Starting solution: repeatedly fill the cheapest remaining cell as much as possible.
Supply TRANSPORT	Units available at each source (row). Total supply is compared with total demand to check the problem is balanced.	**LCM**	Starting solution: repeatedly fill the cheapest remaining cell as much as possible.
Demand	Units required: at each destination (transportation), or units asked for per day (inventory and simulation).	**Vogel**	Starting solution: allocate in the row or column with the largest penalty (gap between its two cheapest costs). Usually closest to optimal.
Cell TRANSPORT	One source-to-destination route. Its small corner number is the unit cost; the big number is the units shipped.	**VAM**	Starting solution: allocate in the row or column with the largest penalty (difference between its two lowest costs).
Empty cell TRANSPORT	A route with no shipment. MODI computes its improvement index $c_{ij} - u_i - v_j$ to see whether using it would lower the cost.	**MODI**	Optimality test for transportation: compute potentials u and v, then improvement indices $c_{ij} - u_i - v_j$ for empty cells. All ≥ 0 means optimal.
Closed loop TRANSPORT	A path of occupied cells that alternates + and − signs, used to shift units without breaking supply or demand totals.	**Hungarian**	Assignment algorithm: subtract row and column minima, cover all zeros with the fewest lines, and adjust until a complete zero assignment exists.
MODI iterations	Rounds of the Modified Distribution method: compute u, v, find the most negative improvement index, reallocate around the loop.	**Maximise profit**	The matrix holds profits; the solver converts it to costs by subtracting each entry from the largest.
NW corner	Starting solution: fill the top-left cell as much as possible, then move right or down. Fast but ignores costs.	**Minimise cost**	The matrix holds costs; the cheapest total assignment is sought.

NETWORK

Edge	A connection between two nodes, with a weight (distance, cost) or a capacity.	**Dijkstra**	Shortest paths from one source when all weights are ≥ 0: repeatedly finalise the closest unfinished node.
Via	The previous node on the shortest path to this node.	**Bellman-Ford**	Shortest paths that allow negative weights; repeats edge relaxations and detects negative cycles.
Distance	Length of the shortest route found from the source to this node.	**Floyd**	Shortest paths between every pair of nodes by allowing one more intermediate node at a time.
Flow	Units currently sent along the edge.	**Kruskal**	Minimum spanning tree: take the lightest edges first, skipping any that would close a cycle.
Capacity	Maximum units the edge can carry.	**Prim**	Minimum spanning tree: grow one tree by always adding the cheapest edge that leaves it.
saturated	An edge carrying its full capacity: no more flow can be pushed through it.	**Max flow**	The most units that can travel from source to sink without exceeding any edge capacity (Edmonds–Karp / Ford–Fulkerson).
min cut	The smallest-capacity set of edges whose removal disconnects source from sink. Its capacity equals the maximum flow.		

PROJECT PLANNING

ES PROJECT	The earliest time the activity can begin, once all its predecessors are finished (forward pass).	**σ²** PROJECT	PERT variance of an activity: ((b − a) / 6)². Variances along the critical path add up.
EF PROJECT	ES + duration: the earliest time the activity can be completed.	**a** PROJECT	PERT: the shortest possible duration if everything goes well.
LS PROJECT	The latest time the activity can begin without delaying the project (backward pass).	**m** PROJECT	PERT: the most probable duration.
LF PROJECT	The latest time the activity can finish without delaying the project.	**b** PROJECT	PERT: the longest duration if things go badly.
Total float PROJECT	LS − ES: how long the activity can be delayed without delaying the project. Zero float means the activity is critical.	**Dn** PROJECT	Duration of the activity at its normal (cheapest) pace.
Free float PROJECT	How long the activity can be delayed without delaying the earliest start of any successor.	**Dc** PROJECT	The shortest possible duration of the activity when fully crashed.
critical / Critical PROJECT	An activity with zero total float. A delay in any critical activity delays the whole project.	**Cn** PROJECT	Cost of the activity at its normal duration.
CPM	Scheduling with fixed activity durations: find the longest path (the critical path), which sets the project length.	**Cc** PROJECT	Cost of the activity when fully crashed.
PERT	Scheduling with uncertain durations: each activity has optimistic, most likely and pessimistic times, giving a mean and variance.	**Activity** PROJECT	A task in the project, with a duration and predecessor activities.
Crashing	Shortening the project by paying extra to speed up activities, choosing the cheapest way each time.	**Duration** PROJECT	How long the activity takes.
te PROJECT	PERT mean duration: (a + 4m + b) / 6.		

QUEUING

ρ	λ / (c·μ): the fraction of time the servers are busy. The queue is stable only if ρ < 1.	**M/M/1**	Poisson arrivals, exponential service, one server, unlimited waiting room.
λ	Average number of customers arriving per unit time.	**M/M/c**	Poisson arrivals, exponential service, c identical servers sharing one queue.
μ	Average number of customers one server can serve per unit time (1/μ is the mean service time).	**M/M/1/N**	One server and room for at most N customers; arrivals who find it full are lost.
L_s / Ls QUEUING	Average number of customers both waiting and being served.	**M/M/c/N**	c servers and room for at most N customers in total.
Lq QUEUING	Average number of customers waiting (not yet in service).	**M/G/1**	Poisson arrivals, one server and any service-time distribution (uses the mean and variance: Pollaczek–Khinchine).
W_s / Ws QUEUING	Average time a customer spends waiting plus being served ($W_s = L_s / \lambda$).	**Machines**	A fixed number K of machines that break down and are repaired by c repairmen.
Wq QUEUING	Average time a customer waits before service begins (W_q = L_q / λ).	**Server cost**	Cost per unit time of keeping c servers.
P_0 QUEUING	Long-run fraction of time with no customers.	**Waiting cost**	Cost per unit time of customers waiting.

INVENTORY

EOQ	The order size that minimises ordering plus holding cost: √(2KD / h).	**Newsvendor**	One-period stocking decision under uncertain demand: stock until the chance of selling the last unit equals the critical ratio.
EPQ	Like EOQ, but stock builds up gradually while production is running.	**Tier**	A quantity range with its own unit price.
Q*	The order (or production) quantity with the lowest total cost.	**Initial cost**	Total cost before the decision is changed.
Shortages	EOQ model that allows backorders at a penalty cost, so stock can run out between orders.	**Final cost**	Total cost after the decision is applied.
Discounts	Unit price falls when you order more; each price break is tested for its own best order size.		

GAMES / MARKOV / SIMULATION

maximin GAMES	The row player picks the row whose worst-case payoff is largest.	**Random number**	A number drawn at random and used to select an outcome in a simulation.
minimax GAMES	The column player picks the column whose best-case payoff for the row player is smallest.	**Cumulative / Cumulative probability**	The running total of probabilities up to and including this value.
dominated GAMES	A strategy that is never better than another one, whatever the opponent does, so it can be removed.	**Interval**	The range of random numbers that maps to this outcome. Its width is proportional to the outcome's probability.
π MARKOV	The long-run fraction of time spent in each state; satisfies $\pi P = \pi$.	**Prob.**	Chance of the outcome, between 0 and 1.
Monte Carlo	Estimate a quantity by repeating a random experiment many times and averaging.	**Mean**	The average value.
Queue sim	Discrete-event simulation of customers arriving and being served, tracked with a clock and an event list.	**Median**	The middle value when results are sorted.
RN	A number drawn at random (for example 00–99) and mapped to a value through the cumulative probability intervals.	**Expected**	The long-run average of the quantity, weighted by probabilities.

GENERAL

from \ to MARKOV	Row = the state you are in now; column = the state you move to next. Each row of probabilities sums to 1.	**A \ B**	Rows are player A's strategies, columns are player B's. Each cell is A's payoff.

MORE LABELS, BY MODULE

in system	Average number of customers waiting plus being served (L_s).	**Payoff matrix**	Gains for the row player (losses for the column player) for each pair of strategies.
in queue	Average number of customers waiting, not yet being served (L_q).	**irreducible**	Every state can eventually be reached from every other state.
time in system	Average time from arrival to departure: waiting plus service (W_s).	**limiting distribution**	The probabilities the chain settles to after many steps, regardless of where it started.
time waiting	Average time a customer waits before service starts (W_q).	**Convergence**	How the state probabilities approach their long-run values as steps increase.
ρ (traffic)	λ / (c·μ): the fraction of time each server is busy. Must be below 1 for an unlimited queue to settle.	**Diagram**	Circles are states; arrows show transition probabilities.
P_0 (empty)	The long-run probability that there are no customers in the system.	**Analysis**	Classification of states, steady state and, for absorbing chains, absorption results.
Server busy	The fraction of time a server is busy serving customers.	**‖∇f‖**	Length of the gradient vector. It is 0 at a stationary point, so small values mean the search has nearly converged.
λ effective	Rate at which customers actually enter the system (arrival rate minus those turned away when it is full).	**f(x)**	Value of the function being minimised at the current point.
P(wait)	The chance an arriving customer has to wait (for M/M/c this is the Erlang C formula).	**step length**	How far the iteration moved along the search direction.
Measures	The standard results of a queue: utilisation, queue lengths, waiting times and probabilities.	**KKT**	Karush–Kuhn–Tucker conditions: necessary conditions for a constrained optimum — feasibility, stationarity of the Lagrangian, non-negative multipliers and complementary slackness.
Cost model	Finds the number of servers that minimises server cost plus customer waiting cost.	**Quadratic**	Minimise a quadratic function $\frac{1}{2}x^TQx + c^Tx$ subject to linear constraints.
Order quantity	How many units to order each time to minimise total cost.	**Unconstrained**	Find the minimum of a function with no constraints (gradient descent or Newton).
Reorder point	Stock level at which a new order is placed so it arrives just as stock runs out (demand during lead time).	**Rule applied**	The named rule or formula used in this step, so you can look it up in your notes.
Cycle length	Time between two orders: Q / D.	**Initialisation / Initialization**	The starting state before the algorithm makes its first move.
Orders / period	How many orders are placed in the time unit (D / Q).	**Tree**	Every sub-problem explored, with its bound and what happened to it.
Ordering	Cost of placing orders: K × number of orders.	**Plane**	The feasible region and integer points drawn in the (x_1, x_2) plane.
Holding	Cost of storing stock: h × average inventory.	**Schedule**	ES, EF, LS, LF, float and critical flag for every activity.
Purchase	Cost of buying the goods: unit cost × demand.	**Gantt**	Bars on a time axis, one per activity or machine.

Policy	The recommended ordering rule and its numbers.	**Network**	Activity-on-node drawing of the precedence relations. Critical activities are highlighted.
Cost curve	Total cost as a function of order quantity; the lowest point is the optimum.	**Allocation**	The shipment plan: how many units go on each route.
Std dev	How spread out the simulated results are around their mean.	**Method comparison**	Starting solutions from NW corner, least-cost and Vogel side by side, with total costs.
Std error	The uncertainty of the estimated mean: standard deviation ÷ √n. Shrinks as you run more trials.	**Solution**	The final answer and the steps that led to it.
5th pct	The value below which 5% of the simulated results fall.	**Heads / Tails**	One of the choices available to a player.
25th pct	The value below which 25% of the simulated results fall.	**Initial**	The starting value.
50th pct	The middle value: half the results are below it.	**Total**	The sum of the components shown above it.
75th pct	The value below which 75% of the simulated results fall.	**Start**	Where the iteration begins.
95th pct	The value below which 95% of the simulated results fall.		

JOB SEQUENCING (NEW)

Sequence SEQUENCING	The order in which the jobs are processed. Every machine uses the same order.	**Busy** SEQUENCING	Total hours the machine is actually working (the sum of its processing times).
Table SEQUENCING	The time each job starts (in) and finishes (out) on every machine.	**Initial wait** SEQUENCING	Time the machine waits at the start before its first job arrives.
Compare SEQUENCING	Total elapsed time from Johnson, CDS and exact search side by side, with the gap to the best.	**Gaps** SEQUENCING	Idle time between two jobs on the same machine.
Data SEQUENCING	The input times for your problem.	**Idle time per machine** SEQUENCING	Total elapsed time minus the machine's busy time: initial wait + gaps + the tail after its last job.
Processing times SEQUENCING	Hours each job needs on each machine.	**Total elapsed** SEQUENCING	Time from the start of the first job until the last job leaves the last machine (also called makespan).
Job SEQUENCING	A piece of work that must go through the machines in order (Machine I first, then Machine II …).	**Gap to optimum** SEQUENCING	How much longer this method's schedule is than the best possible one. 0 means optimal.
Machine I SEQUENCING	The first machine: every job is processed here first.	**Johnson's rule** SEQUENCING	Pick the smallest time among all jobs; if it is on Machine I put the job as early as possible, if on Machine II as late as possible; repeat. Gives the minimum total elapsed time for two machines.
Machine II SEQUENCING	The second machine: a job can start here only after it leaves Machine I and Machine II is free.	**CDS heuristic** SEQUENCING	Campbell–Dudek–Smith: for m machines, apply Johnson's rule to m − 1 combined two-machine versions and keep the best sequence.
Machine SEQUENCING	One of the machines every job passes through.	**Exact search** SEQUENCING	Tries every possible job order (up to 9 jobs) and keeps the shortest.
in SEQUENCING	When the job starts on this machine.	**Gantt chart**	A time chart with one row per machine; blocks are jobs, hatched gaps are idle time.
out SEQUENCING	When the job finishes on this machine.		

SIMULATION (NEW)

Walkthrough SIMULATION	The simulation explained one step at a time; use the player to move through it.	**Unmet demand** SIMULATION	Demand that could not be supplied from stock.
Simulation table SIMULATION	Day by day: the random number, the outcome it selects, and the running results.	**Count**	How many times this outcome occurred.
Summary SIMULATION	Totals and averages of the simulated run compared with the theoretical values.	**First 10 steps of working** SIMULATION	The calculation shown out in full for the first ten numbers so you can follow the method by hand.
Cumulative probabilities SIMULATION	Running total of the probabilities; used to build the random-number intervals.	**n** SIMULATION	Position in the sequence (the seed is step 0).

Probability	Chance of the outcome, between 0 and 1.	**X_n** SIMULATION	The n-th number produced by the generator. X_0 is the seed.
Random numbers (00-99) SIMULATION	Two-digit numbers, each equally likely. The interval an RN falls in decides the simulated outcome.	**X_{n+1}** SIMULATION	Next number, computed from the previous one: $(a \cdot X_n + c) \bmod m$ for the LCG, or the middle digits of X_n^2 for the mid-square method.
Policy search SIMULATION	Tries many order-up-to levels S with the same random numbers and highlights the cheapest.	**u_{n+1}** SIMULATION	The generator state scaled to between 0 and 1 (divide by m).
Totals SIMULATION	Sums over the whole simulated period (costs, units, shortages).	**Formula evaluated** SIMULATION	The generator formula with the actual numbers plugged in.
Setup SIMULATION	The starting conditions and policy of the simulation.	**Cycle and period** SIMULATION	A generator eventually repeats. The period is how many numbers pass before the sequence starts again; longer is better.
Order up to S SIMULATION	At each review, order enough units to bring the stock back up to S.	**Chi-square uniformity test** SIMULATION	Checks whether numbers are spread evenly: counts in equal bins are compared with the expected counts. A very small p-value means the numbers are not uniform.
Review every R SIMULATION	Stock is checked, and an order placed, every R days.	**not uniform** SIMULATION	The test finds the numbers are not evenly spread (p-value below 0.05).
Initial stock SIMULATION	Units on hand at the start of day 1.	**References**	Websites and sources for further reading.

LP: DUALITY, BASIC SOLUTIONS, PRIMAL-DUAL (NEW)

Duality LP	Every linear program (the primal) has a mirror program (the dual). At the optimum their objective values are equal.	**Dual variable** LP	The shadow price of the matching primal constraint.
Basic solutions LP	Every way of choosing basic variables and solving for them. The feasible ones are the corners of the feasible region.	**Dual constraint** LP	The constraint of the dual that comes from a primal variable.
Primal-dual / Primal-dual algorithm LP	A method that keeps a feasible dual solution and improves it until the primal can be solved using only the tight (admissible) columns.	**Type** LP	Whether the constraint is ≤, ≥ or =.
All basic solutions LP	Every choice of m basic columns, solved exactly: feasible, infeasible, degenerate or not a basis.	**Sign** LP	Whether the dual variable must be ≥ 0, ≤ 0 or can be any sign (free).
Basic columns LP	The columns (variables) chosen to be basic in this solution; all others are set to 0.	**Relation** LP	Whether the dual constraint is ≤, ≥ or =.
Corner LP	Marks a feasible basic solution that is also a corner of the feasible polygon (two-variable problems).	**Rules used (SOB table)** LP	Sensible–Odd–Bizarre rules: a memory aid for turning the signs of constraints and variables into the signs of the dual.
infeasible	This basic solution breaks a constraint (some variable is negative).	**Matrix form** LP	The problem written with matrices: max $c^T x$ subject to $Ax \le b$, $x \ge 0$ (and its dual).
optimal	The best feasible value of the objective.	**Both solutions side by side** LP	The optimal primal values (x) next to the optimal dual values (y).
not a basis LP	The chosen columns are linearly dependent, so they cannot be solved for a unique answer.	**z*** LP	The optimal value of the original (primal) objective.
Half-space LP	All points on one side of a hyperplane: where a single inequality holds.	**w*** LP	The optimal value of the dual objective; equals z* (strong duality).
Convex polyhedron LP	The feasible region of an LP: the overlap of half-spaces, with flat faces and corners.	**Duality checks** LP	Exact tests that the solutions are feasible, that the optimal values match, and that complementary slackness holds.
Extreme point (vertex) LP	A corner of the feasible region; it cannot be written as a mix of two other feasible points.	**Pair** LP	A constraint (or variable) together with its matching dual variable (or constraint).

Basic feasible solution (BFS) LP	A basic solution with all variables ≥ 0. Each BFS is an extreme point of the feasible region.	**First factor** LP	The slack (or reduced cost) in a complementary-slackness pair.
Fundamental theorem of LP LP	If an LP has an optimal solution, at least one optimal solution is a basic feasible solution (a corner).	**Second factor** LP	The matching dual variable (or primal variable).
Dual formulation LP	The mirror program: constraints become variables and variables become constraints.	**Product** LP	First factor × second factor. It must be 0 at the optimum.
all checks hold LP	Strong duality, feasibility and complementary slackness were all verified exactly.	**Set-up** LP	The problem written in the standard form the algorithm needs.
Correspondence table LP	Which primal constraint matches which dual variable, and which primal variable matches which dual constraint.	**c (cost)** LP	The coefficients of the objective function.
Primal constraint LP	A constraint of the original problem; each one gives a dual variable.	**OK** LP	This test holds exactly.
Primal variable LP	A variable of the original problem; each one gives a dual constraint.		

SEQUENCING

Johnson	Sequencing n jobs on two machines: pick the smallest time among all jobs; if it is on machine 1 put the job as early as possible, if on machine 2 as late as possible; repeat. Minimises total elapsed time.	**Total elapsed time**	The time from the start of the first job until the last job leaves the last machine.
CDS	Heuristic for m machines: apply Johnson's rule to m − 1 combined two-machine versions and keep the best sequence.	**Idle time / Idle**	Time a machine waits with nothing to process (before its first job, between jobs and after its last job until the end).
Exact	Tries every possible job order (up to 9 jobs) and keeps the best.	**In** SEQUENCING	When the job starts on this machine.
Makespan	The time from the start of the first job until the last job leaves the last machine.	**Out** SEQUENCING	When the job finishes on this machine.

23

23 / TROUBLESHOOTING AND FAQ

Troubleshooting and FAQ

Quick answers when something looks wrong.

SHEET
23 OF 25
REFERENCE

The page says "Something went wrong in this view"

The app shows this message, and the reason, if a view fails. Try these in order: press **Try again**; if the same message returns, press **Reset this module**. This clears only the autosave of that one module (a damaged saved or shared model can cause it). Your named saves stay. If it still fails, press **Back to the overview** and open the module again. A different message, *This screen could not be downloaded*, means the connection dropped or the app was updated while the page was open. Press **Reload**. Your work is autosaved.

My numbers look different from my textbook

Check these causes before you suspect a mistake. The **final answer** should agree; the path may not.

- **Different choice at a tie.** Two variables may tie to enter or leave. Books choose one; the app follows a fixed rule and says when there was a tie. This can give a different pivot order, or a different (equally good) optimal corner.
- **Alternate optima.** The best value is the same, but the point may differ (chapter 21).
- **Different entering rule.** The app picks the most negative reduced cost (Dantzig's rule). Some books use other rules.
- **Minimisation shown as maximisation.** For a minimisation, the tableau may show *z (shown as max −z)*. The sign of the z row is then reversed.
- **Rounding.** The app keeps exact fractions (3/2), while books often round (1.5). Check the decimal value next to the fraction (for example ≈1.5).
- **Starting methods.** NW corner, least cost and Vogel give different starts but the same final cost.
- **Random numbers.** A simulation depends on the seed or the random numbers you typed. Use **Given list** to reproduce a textbook table exactly.
- **Interval convention.** 00–99 versus 01–100 in simulation tables (see walk-through B).

Use **Diff mode** to find where your hand working and the solver first differ.

How do I update to a new version?

The app checks for a new version every time you open it with a connection. If you still see an old version: reload the page; or do a **hard refresh** (`Ctrl` + `Shift` + `R`, or `⌘` + `Shift` + `R` on a Mac); or close all OR/Studio tabs or windows and open it again. For an installed app, close it and open it again while online. If this does not help, clear the site data for the app in the browser settings (this also removes saved models, so download the ones you need first).

Does it work offline?

Yes, after one visit with a connection. Open the app once online first. The app saves itself in the browser. Models, saves and exports all work offline. Only the first load and the check for updates need the internet.

Which browsers are supported?

Any recent version of Chrome, Edge, Firefox, Safari or Brave on a computer, Android or iPhone. JavaScript must be on. Very old browsers (such as Internet Explorer) are not supported. The **PDF** export uses your browser's print window, so allow pop-ups for the site.

Phone tips

- Tap **Show** to open the model; tap **Hide** to see the solution.
- Turn the phone sideways for wide tables and graphs.
- Tap a short form to read its meaning. Tap again to close it.
- Use the menu button (three lines) to change module. Use the step player slider to move between steps.
- Install the app (chapter 2) for a full-screen view.

Is my data private?

Yes. There is no server. Your models are saved only in your own browser, and never sent anywhere. A share link contains the model in the link itself, so only share it with people who may see it.

I lost a model I made

Open the module and look in **Open** (the folder icon). The autosave brings back the last model in the module. Clearing the browser's site data deletes both autosaves and saved models. Download important models as **JSON** files as a backup.

The Graphical tab is missing

That tab is only shown for models with exactly two variables.

Nothing happens when I press a key

You may be typing inside a box. Click on an empty part of the page. A dialog may also be open (Save, Open or Search). Press `Esc` to close it.

I found a mistake in the app

Please report it with the steps to repeat it at the project page: **github.com/Jay-Naik2526/or-studio**.

24

24 / SYLLABUS MAP

Syllabus map

Where each unit of course 702CO0E034 (Operations Research) lives in OR/Studio.

SHEET 24 OF 25 REFERENCE

UNIT AND TOPIC	WHERE IN THE APP
1 · Introduction to OR	The Manual page in the app, and chapter 1 of this guide
2 · LP: formulation, matrix form, extreme points, basic feasible solutions	Linear programming → Model; **Basic solutions** tab
2 · Graphical method and its special cases	Linear programming → **Graphical** tab (library: infeasible, unbounded, alternate optima, degenerate)
2 · Simplex, Big-M, two-phase, dual simplex	Linear programming → **Method** switch
2 · Sensitivity analysis and duality	Linear programming → **Sensitivity**, **Duality** and **Primal–dual** tabs
3 · Gomory cuts, branch and bound	Integer programming
4 · Transportation: NWCR, least cost, VAM, MODI, variations	Transportation
4 · Assignment: Hungarian method, variations	Assignment
5 · CPM, PERT, crashing	Project planning (and Network models)
6 · Two-person zero-sum games	Zero-sum games
7 · Simulation: random numbers, generators, demand tables, inventory systems	Simulation → **Demand table**, **Inventory**, **RNG lab**
7 · Discrete-event simulation (clock, event list), queuing systems	Simulation → **Queue sim**; Queuing analysis
7 · n jobs through two, three and m machines	Job sequencing

E EXPERIMENT 10 (SIMULATION MODELS)

ACTIVITY	FEATURE	LIBRARY PROBLEM
1 · Demand simulation with random numbers	Simulation → **Demand table** (Walk-through B)	`sim-exp10-demand`
2 · Random-number generators and methods	Simulation → **RNG lab**	`sim-rng-lcg` , `sim-rng-midsquare`
4 · Johnson's rule for n jobs on two machines	Job sequencing → **Johnson** (Walk-through C)	`seq-exp10`
5 · Three machines, and job–machine assignment	Job sequencing with three machines; Assignment for the job–machine assignment view	`seq-3m` , `as-3x3`

25

25 / ABOUT

About

Who made OR/Studio and this guide, and how it may be used.

SHEET
25 OF 25
REFERENCE

OR/Studio was **built for the Operations Research course** at **NMIMS Shirpur**, so that every student can see how each method works, step by step, for free.

APP	**OR/Studio**, version 2.0.0. Thirteen solver modules, 273 explained short forms and 66 worked problems.
COURSE	Operations Research, code 702CO0E034, NMIMS Shirpur.
THIS GUIDE	Written by **Dr. Praveenkumar Loharkar**.
LICENCE	**MIT licence.** You may use, copy, change and share the app and its code, free of charge, as long as the licence notice stays.
SOURCE CODE	github.com/Jay-Naik2526/or-studio
LIVE APP	or-studio.antideploy.app
BASED ON	The curriculum of Hamdy A. Taha, *Operations Research: An Introduction*. OR/Studio is not affiliated with the TORA software or its publisher.

1 HOW OR/STUDIO KEEPS ITS ANSWERS RIGHT

- **Exact arithmetic.** Simplex pivots, MODI, Gomory cuts, branch and bound, Markov steady states and game strategies use exact fractions, so 1/3 is stored as 1/3 and never as 0.333.
- **Symbolic M.** Big-M and prohibited routes keep M as a symbol. No large number is ever put in its place.
- **Cross-checked.** The automatic tests compare the solvers against independent methods on thousands of random problems.
- **Honest limits.** It is made for textbook-size problems. Queues, inventory and simulation use ordinary decimal numbers.

2 MADE ACCESSIBLE

Every control can be reached with the keyboard. Step changes are announced to screen readers. Tables have headings. Colours always come with words or icons, and animations respect the *reduce motion* setting of your device.

3 THANK YOU

Thank you to the students and teachers of the course who tried the app and asked for clearer explanations. This guide exists because of your questions.